LANCE et IoTChainBench

Rapport d'ensemble du projet
Objectifs, conception, réalisations et état d'avancement

Projet LANCE
Rédaction par volets spécialisés, coordination Codex

Dernière révision : 2026-10-15 22:53 (Europe/Brussels, UTC+02:00)
État documentaire au 15 octobre 2026 ; résultats historiques attribués

Résumé

LANCE est une plateforme de recherche pour l'audit de sécurité de réseaux d'objets connectés autorisés, développée dans le contexte des travaux UMONS du programme NATO SPS G6259. Le projet associe la représentation du réseau, un moteur d'agents en six phases, une bibliothèque de contrôles, un laboratoire automatisé, une application de suivi, le benchmark IoTChainBench et une chaîne de spécialisation des modèles. Ce rapport présente ces ensembles, leurs interfaces, leurs réalisations et leurs conditions de validation. Six rédactions spécialisées examinent architecture, réseaux, sécurité, application, modèles et évaluation à partir du code et des documents locaux. Les acquis comprennent notamment un graphe NetworkX propre au run, quatre modes d'exécution, des contrats de preuve, 29 scénarios publics, un comparateur de politiques et un service multi-adaptateurs. Des audits historiques ont produit des scores et des diagnostics ; les comparaisons contrôlées restent à achever et aucun avantage du LLM ni gain attribuable à la spécialisation n'est établi par ce bilan. Les qualifications logicielles et les travaux proposés gardent leur statut propre. Les limites majeures concernent la qualification des environnements, la preuve des transitions réseau, l'indépendance des tests, l'attribution des gains aux modèles et l'achèvement des campagnes contrôlées. La feuille de route relie les prochains livrables à des critères de sortie vérifiables. Ce bilan ne vérifie pas en direct le laboratoire et n'ajoute aucune nouvelle expérience d'audit ou d'entraînement.

Table des matières

1 Contexte, finalité et trajectoire du projet **4**
1.1 Un projet d'audit IoT dans un programme de recherche plus large 4
1.2 Le problème : relier une hypothèse à un effet observé 4
1.3 Objectifs complémentaires et critères de réussite 5
1.4 Une évolution du graphe d'analyse vers la plateforme expérimentale 5

2 Méthode du rapport et lecture des états d'avancement **6**
2.1 Six rédactions spécialisées et une intégration commune 6
2.2 Ce que signifie un état annoncé 7

3 Architecture logicielle et moteur d'agents **7**
3.1 Une orchestration aux responsabilités explicites 7
3.2 Représenter le réseau et isoler l'état d'un run 8
3.3 Six phases avec des entrées et des sorties différentes 9
3.4 Boucle modèle–outils, transactions et profils 10

3.5 Maîtriser le contexte et reprendre les productions incomplètes 11
3.6 Connaissances, fournisseurs, ressources et état de ce volet 11

4 Réseaux, laboratoire et infrastructure d'exécution 12
4.1 Un laboratoire reproductible au service de l'audit 12
4.2 Topologies, zones et signification d'un pivot 13
4.3 Familles de services et degré de réalisme 14
4.4 Composition, posture et vérité terrain 15
4.5 Préparation, réservation, contrôle et remise en état 15
4.6 Intégration continue, exploitation et avancement du volet 16

5 Sécurité, confiance et construction de la preuve 17
5.1 Ce que LANCE cherche à établir et à protéger 17
5.2 De l'hypothèse à une observation attribuable 17
5.3 Des critères adaptés aux protocoles et aux propriétés 18
5.4 Confirmation, résultat négatif et résultat indéterminé 19
5.5 Observation de service, accès obtenu et pivot réseau 19
5.6 Sécurité du moteur et de l'application de pilotage 20
5.7 Acquis, limites et prochaines étapes de sécurité 20

6 Application, expérience opérateur et exploitation 21
6.1 Transformer le moteur d'audit en un parcours utilisable 21
6.2 Des modes et profils qui répondent à des besoins différents 22
6.3 Configuration des modèles et préconditions du lancement 23
6.4 Coordination, événements et continuité du suivi 23
6.5 Topologie et résultats : montrer ce que les données permettent 24
6.6 Historique, comparaison et diagnostic des ressources 24
6.7 Accès administratif, acquis et travail restant 25

7 Modèles, données et spécialisation par apprentissage 25
7.1 Pourquoi spécialiser un modèle local ? 25
7.2 QLoRA : une base quantifiée et des corrections spécialisées 26
7.3 Quatre experts et un routage déterministe par phase 26
7.4 Origine des corpus et préparation des conversations 27
7.5 Entraînement, validation et reproductibilité 28
7.6 Service d'inférence et limites de concurrence 28
7.7 Boucle de correction supervisée et protection de l'évaluation 29
7.8 État démontrable et expériences encore nécessaires 29

8 Benchmark, méthode scientifique et résultats disponibles 30
8.1 Un corpus public composé, dont le test reste provisoire 30
8.2 Mesurer des populations, des preuves et des accès distincts 30
8.3 Attribuer un éventuel gain à une politique de décision 31
8.4 Appariement, budgets et prudence statistique 32
8.5 Campagnes historiques : résultats et défauts observés 33

8.6 Pilote D1/A1 : budgets calibrés, campagne non renseignée 34
8.7 Plan V2 : rendre les questions et les mesures explicites 34
8.8 Premier incrément SSH et conditions de passage à la mesure 35

9 Avancement transversal et portefeuille de recherche 36
9.1 Une progression à lire par livrables 36
9.2 Les recherches qui accompagnent le développement 36

10 Dépendances, limites et priorités de poursuite 37
10.1 Le chemin critique relie environnement, preuve et décision 37
10.2 Un programme de travail par critères de sortie 37
10.3 Publication et diffusion 38

11 Conclusion générale 38

Références 39

A Repères de vocabulaire 44

1 Contexte, finalité et trajectoire du projet

1.1 Un projet d'audit IoT dans un programme de recherche plus large

Le projet LANCE étudie l'automatisation d'audits de sécurité de réseaux d'objets connectés. IoT signifie *Internet of Things* ; OT, *Operational Technology*, désigne ici les composants et services liés aux systèmes industriels. Une infrastructure de ville intelligente peut associer capteurs, passerelles, équipements de réseau, courtiers de messages, services applicatifs et systèmes de supervision. Sa sécurité dépend à la fois des propriétés de ces composants et des relations qui les rendent accessibles.

Le document local préparé pour le jalon III du programme NATO SPS G6259, *Enhancing IoT Systems Security*, situe le travail de l'Université de Mons dans les activités d'expérimentation virtuelle, d'évaluation automatisée et de méthodologie de sécurité IoT/OT. Il décrit une plateforme Proxmox distincte du testbed physique des partenaires, ainsi que le pipeline LANCE et le benchmark IoTChainBench. Ce document daté du 1er septembre 2026 donne le contexte institutionnel ; il est utilisé comme déclaration de suivi, sans supposer une validation indépendante de tous les résultats qu'il mentionne [1].

Le présent rapport développe le périmètre technique disponible dans le dépôt : représentation du réseau et des connaissances, moteur d'agents, interfaces, laboratoire, contrôles de sécurité, mesure expérimentale et spécialisation des modèles. Les activités des autres partenaires fournissent le contexte du programme. Leur avancement détaillé, leurs données financières et leurs résultats propres relèvent de leurs documents de suivi.

Trois ensembles ont des fonctions différentes : LANCE conduit et restitue l'audit ; IoTChainBench fournit des situations de test et leurs références pour l'évaluer ; la chaîne d'entraînement prépare des variantes spécialisées des modèles. Un audit peut employer un modèle déjà disponible auprès d'un fournisseur, sans lancer d'entraînement. Le laboratoire sert à reconstruire des situations connues ; sur un réseau existant, les observations restent exploitables, mais l'absence de référence exhaustive empêche de mesurer toutes les failles manquées [2, 31, 68].

1.2 Le problème : relier une hypothèse à un effet observé

Un audit utile doit dépasser l'énumération des ports ouverts et des versions logicielles. Il doit déterminer quels services sont atteignables, quels droits sont nécessaires, quels comportements sont réellement observés et quelles conclusions restent indéterminées. Deux équipements portant le même logiciel peuvent présenter des situations différentes selon leur configuration, leur exposition réseau, l'identité utilisée ou l'état d'une session.

Les outils spécialisés savent effectuer des contrôles précis, mais leur enchaînement et l'interprétation des résultats demandent un raisonnement sur le contexte. Un modèle de langage de grande taille, ou LLM (*Large Language Model*), peut proposer des hypothèses et des actions à partir d'observations hétérogènes. Il peut aussi produire une explication plausible sans preuve suffisante, ignorer une erreur d'outil ou dépasser le budget avant de vérifier une piste. Le projet vise donc à rendre ses décisions observables et contestables, plutôt qu'à traiter sa réponse comme une vérité de sécurité [2, 3].

LANCE signifie *LLM Agent for Network Compromise Evaluation*. Le modèle intervient dans une chaîne dont les autres éléments conservent des responsabilités propres : les sondes observent, les contrats bornent la preuve, le moteur orchestre et comptabilise, le benchmark confronte aux propriétés attendues et le rapport rend les résultats consultables. Cette répartition est centrale pour comprendre les réalisations du projet.

Une exécution, appelée *run* dans le logiciel, regroupe une configuration de départ, un périmètre autorisé, les actions réalisées et leurs fichiers de résultat. L'agent est la boucle logicielle qui donne au modèle des observations, reçoit ses propositions et fait exécuter les outils admis. Le modèle ne contacte pas directement les équipements : le moteur traduit ses demandes en appels contrôlés.

Cette unité permet de relire une décision dans son contexte, de distinguer deux essais et de conserver aussi les tentatives incomplètes [9, 10].

Concrètement, dans un scénario de laboratoire, l'opérateur choisit le réseau, le modèle et les budgets, puis lance l'audit. Le moteur prépare l'environnement, examine la topologie, recherche les services, analyse les faiblesses possibles, vérifie les hypothèses et évalue les accès. Il conserve ensuite un rapport lisible, les résultats structurés, les traces d'outils et la consommation. L'opérateur peut examiner une confirmation, une tentative non concluante ou une étape interrompue à partir de ces éléments. Ce parcours décrit le fonctionnement prévu par le code ; il ne suppose ni découverte exhaustive ni réussite de chaque phase [9, 27, 82].

1.3 Objectifs complémentaires et critères de réussite

Le projet possède plusieurs objectifs liés, mais évaluables séparément. Le tableau 1 les associe à un livrable et à une condition de validation. Cette lecture évite qu'une interface aboutie, un modèle entraîné ou un scénario défini soit utilisé comme preuve de toutes les autres dimensions.

Objectif	Production attendue	Ce qui permet d'en apprécier la réussite
Comprendre le réseau	Représentation des équipements, services, liens et chemins potentiels.	Provenance des informations et séparation entre topologie déclarée et connectivité observée.
Conduire un audit	Pipeline, outils, états et rapport.	Exécution traçable, résultats soutenus et échecs explicitement conservés.
Expérimenter	Laboratoire, scénarios, témoins et remises à zéro.	État initial vérifié et conditions reproductibles.
Mesurer	Vérités terrain, contrats probatoires et comparateurs.	Unités, dénominateurs, exclusions et versions connus.
Spécialiser les modèles	Données relues, adaptateurs, service et routage.	Gain attribuable à la spécialisation sur des évaluations appropriées.
Rendre le travail utilisable	API, tableau de bord, documentation et artefacts.	Parcours compréhensible et accès aux observations nécessaires à la décision.

Table 1 – Les objectifs couvrent un instrument logiciel, une infrastructure expérimentale et une démarche scientifique. Un même livrable peut contribuer à plusieurs objectifs sans suffire à les valider tous.

La question scientifique actuelle demande **dans quelles conditions et à quel coût une politique avec LLM améliore l'audit face à une automatisation conditionnelle compétente**. Elle porte sur les détections étayées, les faux positifs, la fiabilité, l'adaptation et la consommation. Une question complémentaire concerne la spécialisation d'un modèle local : un routage par rôle et des adaptateurs entraînés améliorent-ils le comportement au-delà du modèle de base et du seul guidage par prompts ? Ces questions appellent des comparaisons distinctes et ne présument pas une réponse favorable [3, 4].

Une politique est la règle qui choisit la prochaine action à partir des observations disponibles. Une automatisation conditionnelle peut détecter un port web, puis lancer les contrôles HTTP prévus pour ce service. Une politique avec LLM peut proposer une autre vérification en interprétant la réponse reçue. La comparaison examine si cette liberté améliore les résultats suffisamment pour justifier ses appels au modèle, ses délais et ses erreurs. La présence d'une explication en langage naturel ne constitue pas à elle seule le gain recherché.

1.4 Une évolution du graphe d'analyse vers la plateforme expérimentale

Les documents historiques permettent de situer la trajectoire sans transformer leurs affirmations anciennes en état actuel. Le rapport Q4 2025 décrit une approche centrée sur Neo4j, un

enrichissement par la base de vulnérabilités NVD et une interaction en langage naturel. Le rapport Q1 2026 décrit ensuite le passage à une infrastructure déclarative YAML, à un graphe NetworkX en mémoire et à un pipeline d'agents. Le graphe courant est construit en mémoire avec NetworkX et chaque exécution conserve son propre état ; le pipeline coordonne les actions des phases et leurs productions. Les anciens serveurs et commandes documentent cette trajectoire sans être supposés nécessaires au système actuel [5, 6].

Les documents de mission placent également la spécialisation des modèles locaux au centre d'un volet de travail : préparation des traces, adaptation QLoRA, service multi-adaptateurs et routage par rôle. Le pipeline et le laboratoire servent alors de cadre pour vérifier le comportement de ces modèles. L'évolution récente de septembre et octobre 2026 renforce les contrats de preuve, l'isolation des runs, l'analyse des échecs et les comparaisons avec des politiques sans LLM [4, 7, 8].

Cette trajectoire explique pourquoi le projet comprend plus qu'un agent conversationnel : il associe un moteur, un modèle de réseau, une bibliothèque de contrôles, une application de suivi, un banc d'essai et un outillage d'apprentissage.

La figure 1 relie ces ensembles avant leur description détaillée. Elle distingue le moteur qui décide et coordonne, les moyens d'observation, les productions consultables et les retours d'apprentissage.

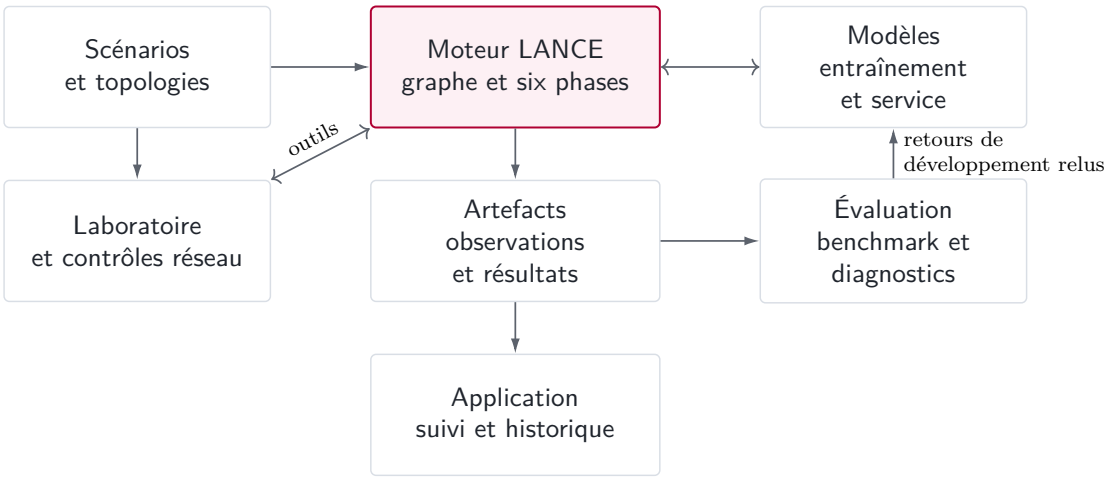


Figure 1 – Vue d'ensemble du projet et des flux d'information. Les retours vers l'apprentissage exigent une acceptation humaine et concernent les données de développement autorisées. Les scénarios de test restent exclus de ce flux.

2 Méthode du rapport et lecture des états d'avancement

2.1 Six rédactions spécialisées et une intégration commune

La révision répartit la rédaction entre six regards : architecture logicielle, réseaux et infrastructure, sécurité et preuves, application et usages, benchmark et méthode scientifique, modèles et entraînement. Chaque rédacteur examine les sources de son domaine, produit une partie développée et identifie ses références. L'intégration harmonise le vocabulaire, la progression et les limites ; les explications nécessaires sont données au lieu de leur emploi. La relecture croisée et l'inspection du PDF vérifient ensuite la cohérence du document dans son ensemble.

La copie locale est fondée sur le commit `73369b03359d4facc8ec58a233c1623284e746d0`, avec des modifications et des fichiers non versionnés. Le manifeste `sources-locales.json` conserve les chemins et empreintes des sources utilisées. Les résultats sont attribués à leurs comptes rendus, avec leurs dates. Aucune nouvelle campagne réseau, connexion au laboratoire, génération par un fournisseur externe ni session d'entraînement n'est lancée pour rédiger ce rapport.

2.2 Ce que signifie un état annoncé

Un composant *implémenté* est présent dans le code et ses chemins d'exécution inspectés. Une *validation logicielle* exerce un comportement dans des conditions définies, souvent avec outils ou réponses simulés. Une *observation de laboratoire* décrit un essai et son environnement. Une *proposition* décrit un travail à réaliser. Ces états apparaissent dans les bilans des différentes parties.

Un contrat décrit des conditions explicites que le logiciel peut vérifier : champs requis d'un résultat, observations admissibles pour une propriété, règles de calcul ou critères de fin d'une phase. Il relie une déclaration à des contrôles reproductibles. Annoncer une lecture MQTT anonyme demande, par exemple, une observation de l'échange applicatif pertinent et de son résultat. Le contrat reste limité aux cas qu'il reconnaît ; sa version est conservée, car une évolution des règles peut changer l'interprétation du même journal [10, 65].

Le rapport évalue la maturité par livrables et par preuves, sans calculer un pourcentage global arbitraire. Un scénario défini n'est pas nécessairement déployé ; un run achevé n'a pas nécessairement couvert toutes les propriétés ; un score ancien ne devient pas comparable à un score nouveau après changement de contrat. Les termes *confirmé*, *terminé* et *validé* gardent donc le périmètre qui leur donne un sens.

Les publications et anciens rapports donnent une provenance aux orientations du projet. Ils ne remplacent pas la lecture du code courant et de ses artefacts. Les limites d'accès ou de reproduction sont signalées ; une inconnue reste une inconnue dans les tableaux d'avancement.

3 Architecture logicielle et moteur d'agents

3.1 Une orchestration aux responsabilités explicites

LANCE organise l'audit autour d'un moteur qui possède le cycle de vie de l'exécution. Il prépare les ressources, choisit les phases applicables, vérifie leurs prérequis et conserve un état terminal. Le modèle intervient dans les décisions qui lui sont déléguées ; il ne décide pas seul qu'un fichier est valide, qu'un budget est disponible ou qu'une phase est terminée. Cette séparation permet de modifier un modèle ou un prompt sans déplacer automatiquement les responsabilités du contrôleur [9, 10].

L'évolution du socle explique cette organisation. Le rapport du quatrième trimestre 2025 décrivait Neo4j, MongoDB et une interrogation via MCP ; celui du premier trimestre 2026 présentait le passage à YAML, NetworkX et ChromaDB. Ce sont des descriptions historiques. Dans le code inspecté au 15 octobre 2026, le backend de graphe implémenté est NetworkX ; SQLite conserve notamment le catalogue des fournisseurs et modèles, les runs et leur consommation. L'abstraction du graphe autorise une autre implémentation, mais ne constitue pas un adaptateur Neo4j opérationnel [5, 6, 11, 12].

La figure 2 situe les responsabilités actuelles. Le pipeline appelle des modules de phase explicitement enregistrés, sans découverte de plugins. Un contexte de phase transmet les données et services nécessaires. La boucle modèle–outils dialogue avec le fournisseur ; l'exécuteur applique les contrôles communs et archive les observations. Le rapport et les phases d'agrégation peuvent également travailler sans cette boucle lorsqu'une transformation déterministe suffit. Le code conserve néanmoins des adaptateurs et méthodes de compatibilité dans `Pipeline` : la séparation en modules n'équivaut pas à six services indépendants [13, 43].

Concrètement, l'initialisation crée un dossier identifié, enregistre la configuration et la révision du code, puis prépare la topologie ou le contexte de découverte. Les phases retenues sont ordonnées ; sélectionner une phase aval peut ajouter ses prérequis. Le moteur vérifie aussi la présence réelle des livrables nécessaires : demander un rapport ne permet pas de contourner une vérification absente. Après exécution ou interruption, un bloc de finalisation tente de figer les consommations, effectuer le nettoyage prévu et écrire les métadonnées terminales. L'échec d'une opération de finalisation

reste distinct de la cause initiale ; une conservation incomplète des mesures peut dégrader le statut. Le résultat décrit ainsi le déroulement effectivement enregistré, plutôt qu'une simple arrivée à la dernière instruction [9].

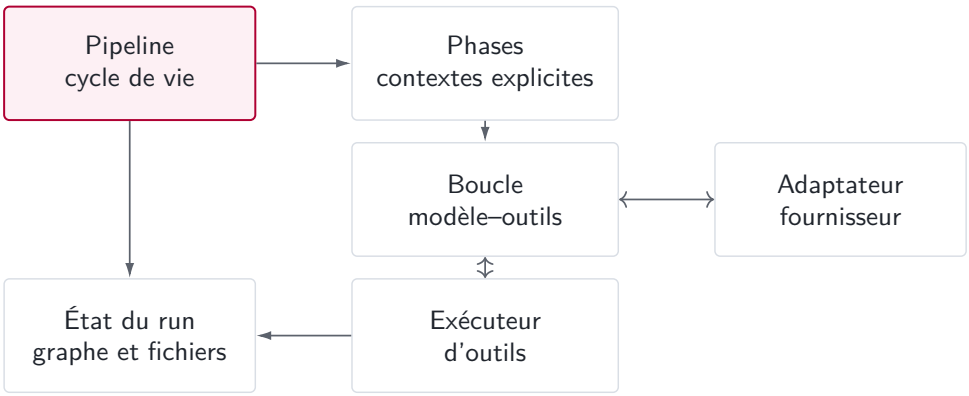


Figure 2 – Le pipeline possède le cycle de vie et l'état du run. Les phases emploient les services du moteur ; les observations et fichiers restent rattachés à cette exécution. Les flèches représentent des appels et échanges, pas une séparation entre machines [13, 32].

3.2 Représenter le réseau et isoler l'état d'un run

Le graphe distingue les équipements, leurs attributs et les relations de connectivité. Le chargeur YAML construit des objets d'infrastructure puis un `DiGraph` NetworkX. L'interface du backend propose consultation d'un équipement, voisinage, surface exposée, chemins et export sérialisable. Chaque opération possède sa sémantique : la recherche générique de chemins utilise une vue non orientée, tandis que le calcul de risque et l'analyse pondérée des attaques respectent la direction du graphe. Un chemin de connectivité reste une propriété du modèle, sans prouver un passage exploité [15, 11, 16, 17].

Dans ce graphe, un nœud représente un équipement et un arc une relation de connectivité déclarée. L'orientation distingue un passage de A vers B du passage inverse ; elle importe lorsque routes et filtrage rendent les accès asymétriques. La centralité d'intermédiarité souligne les équipements situés sur de nombreux chemins calculés, par exemple une passerelle reliant plusieurs zones. Elle aide à prioriser l'investigation. Une recherche générique qui ignore l'orientation explore plutôt la structure des connexions ; son résultat ne doit pas être interprété comme une autorisation de passage dans les deux sens.

Le vocabulaire d'enrichissement distingue quatre objets. NVD (*National Vulnerability Database*) est la base consultée ; CVE (*Common Vulnerabilities and Exposures*) identifie une vulnérabilité publiée ; CPE (*Common Platform Enumeration*) décrit un produit et sa plateforme ; CVSS (*Common Vulnerability Scoring System*) cote la sévérité. Un identifiant CVE connu ne prouve pas son applicabilité à un équipement.

Le parcours d'infrastructure déclarée peut enrichir les équipements à partir de correspondances CPE et des données NVD. Le score de risque, borné entre 0 et 10, combine 0,4 fois la composante vulnérabilités, 0,3 fois l'exposition et 0,3 fois la centralité. L'exposition dépend des services et de la distance dirigée depuis le nœud Internet ; la centralité repose sur l'intermédiarité dirigée. Ces poids constituent la politique heuristique configurée `directed-v2`. L'analyse des chemins emploie des coûts dérivés de facteurs protocolaires et d'exploitabilité ; une représentation pondérée permet aussi de comparer des distances après modification de l'état. Ces calculs sont des heuristiques d'aide à la priorisation. Ils ne mesurent ni une probabilité de compromission calibrée ni un pivot observé [18, 16, 17, 19].

Le comportement change selon l'entrée. Un scénario public fournit une topologie et des services déclarés, sans précharger ses réponses d'évaluation : les outils de risque et de chemins signalent alors que ces résultats ne sont pas précalculés. Le mode découverte commence avec un contexte minimal et l'enrichit par reconnaissance. Les outils exposent une topologie résumée, des détails par équipement et une surface exploitable par les phases suivantes [20].

Chaque `RunContext` possède son dossier et son `GraphContext`. Un contexte d'exécution lie ce graphe aux appels, y compris dans les workers. Les variables de phase sont copiées et présentées comme une correspondance non modifiable ; des services explicites conservent les opérations autorisées. Cette isolation évite de confondre deux runs, sans constituer un bac à sable système. Les appels autonomes d'outils de graphe conservent une session de compatibilité distincte ; le pipeline emploie son propre graphe [13, 21].

3.3 Six phases avec des entrées et des sorties différentes

Phase 1 : analyse de la représentation disponible. L'entrée comprend le périmètre et la topologie connue, éventuellement presque vide en découverte. Le modèle consulte notamment la topologie, la surface déclarée et les détails d'équipements pour hiérarchiser l'exploration. Il produit `01_graph_analysis.md` ; une projection conserve les observations des outils de graphe. La phase distingue les propriétés déclarées des observations réseau qui restent à réaliser. Elle ne remplace donc pas la reconnaissance [14, 20].

Phase 2 : reconnaissance et couverture. À partir du périmètre et de l'analyse initiale, l'agent choisit des sondes ARP, Nmap et les outils disponibles. Il recherche les hôtes, ports et versions, puis confronte ces observations aux services attendus. Le journal alimente `02_recon_evidence.json` et la restitution `02_recon.md`. Le contrôleur vérifie la couverture requise avant sauvegarde. Une version observée complète le graphe ; une sonde en échec reste signalée. La couverture de ce contrat ne garantit pas la découverte de tous les services possibles [22, 7].

ARP recherche des équipements sur le segment local en reliant adresses IP et adresses matérielles ; il ne découvre pas directement toutes les machines derrière un routeur. Nmap interroge des destinations et ports pour identifier les services joignables et, selon la sonde, des informations de version. La couverture confronte les contrôles requis aux observations enregistrées : un service SSH déclaré mais jamais interrogé reste un manque de couverture ; une sonde exécutée en erreur reste un échec visible. Cette comptabilité distingue le travail tenté des observations effectivement obtenues.

Phase 3 : observation puis analyse par équipement. Les scanners recueillent d'abord les résultats. Chaque analyste reçoit un contexte d'appareil et peut employer des contrôles protocolaires bornés, par exemple HTTP, TLS ou une recherche CVE ; aucun shell général n'est nécessaire à ce parcours. Les résultats par appareil alimentent une agrégation déterministe qui normalise, déduplique et conserve la provenance. Le livrable canonique `03_vuln_analysis.json` et le registre brut restent distincts ; une exclusion ne supprime pas la proposition originale. Les échecs des workers, c'est-à-dire des tâches d'analyse exécutées séparément, restent visibles dans le statut de phase [23, 24].

Phase 4 : contrôles ciblés des hypothèses. La file d'analyse détermine les tâches de vérification. Des workers sélectionnent les outils pertinents, par exemple une requête HTTP ou un contrôle d'identifiants, selon la cible et le contrat applicable. Les observations attribuées aux tâches sont agrégées dans `04_exploitation.json`. Le code reconstruit les statuts depuis les traces, indépendamment du verdict écrit par le modèle. La découverte d'autres hôtes peut déclencher un suivi d'analyse dans le parcours général ; le parcours comparatif borné conserve son périmètre propre [25, 10].

Phase 5 : actions et évaluation des accès. Le moteur prépare les points d'entrée, cibles et informations récupérées. L'agent peut tester un accès, employer un identifiant ou réaliser une opération autorisée sur un hôte. En profil complet, un marqueur de fin demande au contrôleur de construire `05_intrusion.json` depuis le journal ; le modèle ne réécrit pas toute la campagne. Une garde évite une clôture sans tentative lorsque des entrées existent. Elle n'exige pas une réussite, et la projection actuelle ne reconstruit pas les transitions réseau [26, 7].

Sur un même exemple SSH, la phase 4 cherche à confirmer un constat précis, tel qu'un identifiant faible permettant une connexion. La phase 5 reçoit les points d'entrée et informations récupérées, puis examine les accès et opérations qu'ils rendent possibles dans le périmètre autorisé. Elle peut donc réemployer un identifiant pour observer un accès effectif ; elle ne constitue pas une seconde mesure de détection, et une connexion directe ne prouve pas un pivot.

Phase 6 : restitution à partir des faits. Les hypothèses, vérifications et observations d'accès deviennent des fiches. Le modèle rédige leurs commentaires et recommandations sans outils, dans un contexte borné. Le code assemble les faits et calcule la synthèse ; les commentaires demeurent signalés comme non validés. Le rapport `06_report.md` conserve les inconnues et les défauts de rédaction. Les fiches déjà utilisables peuvent être réemployées si leurs sources, prompt, modèle et réglages sont identiques. Cette reprise ne relance aucun contrôle réseau [27, 28].

Le livrable `06_report.md` est un document Markdown consultable dans l'application. Il réunit une synthèse, des tableaux factuels, les fiches des hypothèses, les observations d'accès et les limites. Une fiche sans test ou non concluante conserve cet état ; les fichiers structurés et le journal `tool_calls.jsonl` permettent d'en examiner les sources.

3.4 Boucle modèle–outils, transactions et profils

Une phase construit son prompt, la consigne transmise au modèle, à partir de modèles de texte, d'inclusions communes et des variables du run. Le catalogue décrit les outils par nom, description et schéma d'arguments ; les définitions YAML peuvent produire des adaptateurs de sous-processus ou référencer un gestionnaire Python. Les définitions matérielles fournissent des propositions de commandes, sans prouver qu'un équipement physique a été piloté. Les outils indisponibles sont filtrés avant présentation au modèle [29, 30].

La boucle transmet l'historique au fournisseur, reçoit une réponse ou des appels structurés, exécute les fonctions sélectionnées et réinjecte leurs résultats. L'exécuteur commun attribue un identifiant, une cible, une durée et une origine aux observations. Une sauvegarde est transactionnelle : la soumission est archivée, validée puis promue ; un rejet revient au modèle comme erreur d'outil. Un texte final sans sauvegarde acceptée ne suffit pas. Les limites de tours et les gardes contre répétitions ou réponses sans action bornent cette conversation [31, 32, 10].

Pour un service web, le modèle peut demander une requête vers une adresse, un port et un chemin précis au moyen d'arguments structurés. L'exécuteur vérifie ces arguments, appelle le client HTTP et conserve la réponse. Le modèle reçoit alors une observation sur laquelle appuyer la décision suivante. S'il propose un constat, sa sauvegarde doit satisfaire le schéma et les règles applicables. La demande d'outil, sa réponse et le constat qui l'interprète sont trois objets distincts. Ce découpage aide à distinguer une erreur de décision, une erreur d'outil et un refus du service audité.

Le tableau 2 précise trois choix souvent confondus. `auto` résout un profil ; il n'ajoute pas un troisième moteur. La priorité va au choix explicite du run, puis au réglage du modèle, puis aux paramètres actifs ou totaux connus. Le seuil courant est de 32 milliards : au plus ce seuil, le profil est compact ; sans métadonnées, il est complet. Cette règle d'ingénierie n'est pas une mesure de capacité du modèle [33]. Par exemple, les plafonds courants de phase 1 sont 20 tours et 4 096 jetons de sortie par réponse en complet, contre 12 tours et 2 048 jetons en compact ; ils ne décrivent pas la fenêtre de contexte effectivement acceptée par le serveur.

Choix	Effet sur l'exécution	Limite d'interprétation
`full`	Surface autorisée par phase, budgets complets ; choix d'outils plus étendu.	Aucun privilège supplémentaire au-delà du périmètre du run.
`compact`	Outils routés et budgets adaptés ; observations présentées sous forme réduite.	La réduction peut retirer du contexte utile ; les sources annexes restent nécessaires.
`auto`	Résolution traçable depuis le réglage et les métadonnées du modèle.	Le nombre de paramètres ne garantit ni compétence ni fiabilité.

Table 2 – Les profils modifient la présentation et les ressources, sans changer la vérité terrain ni l'évaluateur [33].

3.5 Maîtriser le contexte et reprendre les productions incomplètes

L'enjeu n'est pas seulement la longueur d'un prompt initial. Une conversation réexpédie aussi les résultats précédents ; multiplier équipements et sondes augmente le texte traité et le risque de restitution incomplète. Le code réduit ce couplage par des contextes spécialisés. La phase 3 sépare scan, analyse d'appareil et agrégation. La reconnaissance peut, pour une topologie déclarée éligible, séparer un balayage initial et des lots de couverture, puis les fusionner depuis le journal. Le mode aveugle commence sans topologie fournie ; le mode scellé ajoute une séparation technique entre l'agent et les réponses de l'évaluateur. Ces modes conservent le parcours simple [23, 34].

Plusieurs récupérations répondent à des signatures précises. Une phase 1 complète tronquée peut reprendre seulement la rédaction depuis les faits de graphe enregistrés. Pour une analyse d'appareil tronquée, la phase 3 peut répartir la restitution en blocs de services, sauvegardés séparément avant assemblage. Le contrôleur refuse la promotion si un bloc requis manque. La phase 6 retente une fiche tronquée, puis conserve explicitement son incomplétude. Les reprises restent bornées ; elles ne certifient pas la capacité effective du serveur distant [35, 36, 27].

Exemple de fonctionnement, sans résultat expérimental. Considérons un équipement exposant SSH, HTTPS et MQTT. Les sondes peuvent avoir abouti alors que la réponse d'analyse s'interrompt avant sauvegarde. La récupération par blocs permet de traiter ces observations dans des productions plus petites, sans réexécuter les sondes. Si la restitution MQTT reste invalide, l'appareil demeure incomplet ; deux blocs acceptés ne deviennent pas automatiquement une analyse complète. Le bénéfice attendu est de localiser l'échec et de conserver le travail réutilisable, pas de garantir moins de coût ou de faux négatifs [36].

3.6 Connaissances, fournisseurs, ressources et état de ce volet

La mémoire comprend plusieurs objets différents. Les fichiers du run assurent la continuité entre phases. Les fiches méthodologiques Markdown peuvent être chargées selon leurs étiquettes. ChromaDB fournit une recherche sémantique sur connaissances et historiques. Un embedding représente un texte par un vecteur numérique afin de retrouver des contenus proches ; sa production utilise ici le client Voyage. L'emploi de cette recherche dépend des services et identifiants configurés. Une similarité textuelle n'établit pas qu'une CVE concerne la version observée : une classification CPE séparée conserve les cas compatibles, conditionnels, indéterminés et incompatibles [37, 18]. Le stockage vectoriel prévoit un repli éphémère si son ouverture persistante échoue ; disposer de l'API de recherche ne garantit donc pas, à lui seul, la conservation durable d'une mémoire [42].

Ces mécanismes changent dans le benchmark. La recherche CVE utilise un instantané versionné, sans consulter le cache mutable ni interroger NVD en cas d'absence. La recherche d'historique et les collections persistantes hors méthodologie sont refusées ; les résultats du benchmark ne sont pas réinjectés en mémoire épisodique. Hors de ce cadre, le parcours général conserve une recherche en cache puis une interrogation distante. La reproductibilité dépend donc aussi du mode de connaissances, pas seulement du nom du modèle [37, 38, 32].

L'interface fournisseur unifie des endpoints compatibles avec le format OpenAI, configurés notamment dans SQLite. Un modèle peut être choisi par phase. Le contrôle d'admission sérialise, lorsque configuré, les requêtes partageant une origine ; les outils ne gardent pas ce verrou. Cela aide un serveur local contraint, sans créer de capacité GPU. Le transport distingue les reprises d'une même requête des continuations du raisonnement : une reprise réseau ne rejoue pas elle-même les outils [31, 39, 40].

Le suivi agrège jetons, appels, erreurs, temps des agents et temps écoulé. Un jeton est une unité de texte traitée par le modèle : mot, fragment de mot ou symbole. Des observations réexpédiées dans plusieurs requêtes peuvent donc être comptabilisées plusieurs fois. Les tarifs retenus sont conservés par phase avec leur provenance et leur caractère estimé. La limite financière porte sur la consommation observée : des appels déjà engagés peuvent encore aboutir. Elle ne réserve pas à l'avance toute facture future et ne représente pas le coût matériel d'un modèle local. La finalisation tente de préserver consommation, cause d'arrêt et diagnostics, même lorsqu'une étape échoue [41, 9].

Les acquis de ce volet sont un moteur commun, des contextes propres au run, des sorties structurées et plusieurs reprises ciblées. Ses limites restent la dépendance au comportement des modèles, à la disponibilité des outils et aux ressources de calcul ; le découpage ne démontre pas une scalabilité générale. Les prochaines étapes proposées sont de stabiliser les contrats de contexte, qualifier leur couverture et comparer la fiabilité des profils sur des charges définies. Elles devront préserver les statuts incomplets et les coûts des tentatives, plutôt que compter seulement les rapports produits.

4 Réseaux, laboratoire et infrastructure d'exécution

4.1 Un laboratoire reproductible au service de l'audit

Le volet réseau de LANCE fournit l'environnement dans lequel les décisions de l'agent deviennent des observations vérifiables. Il doit rendre possible une question concrète : après avoir réinstallé un scénario donné, le même service, la même exposition et les mêmes restrictions d'accès sont-ils présents ? Sans cette maîtrise, une absence de résultat pourrait provenir du laboratoire plutôt que de la méthode d'audit. L'infrastructure constitue donc une partie de l'instrument scientifique, avec ses propres contrats et limites.

La cible retenue est le laboratoire **nato / pve-nato**. Proxmox héberge les charges expérimentales et le contrôleur nommé `nato-master`. Les playbooks distinguent les conteneurs LXC Debian, utilisés pour les services, et une machine virtuelle OpenWrt, utilisée comme routeur de scénario. Des modèles préalablement préparés servent de base au clonage. Cette approche réduit le coût de reconstruction et permet de combiner plusieurs rôles sans disposer d'un équipement physique par rôle. Le contrôleur lui-même est provisionné comme conteneur LXC ; le terme historique de « VM maître » ne désigne donc pas une seconde machine physique [45, 46].

Proxmox fournit la couche de virtualisation qui héberge les charges du scénario. Un conteneur LXC isole un environnement Linux en partageant le noyau de l'hôte ; une machine virtuelle possède son propre système, comme pour le rôle OpenWrt. Ansible applique les procédures de création et de configuration décrites dans des playbooks. Le bridge joue le rôle de commutateur virtuel entre les interfaces. Cette chaîne permet de décrire un laboratoire dans des fichiers puis de le reconstruire ; sa répétabilité dépend des modèles système, des versions de paquets et des contrôles d'état initial.

Trois responsabilités doivent être distinguées. Le *plan de commande* correspond à l'opérateur, à l'API et aux workflows qui demandent une opération. Le *contrôleur* exécute LANCE, appelle Ansible et dialogue avec Proxmox pour créer, configurer et retirer les charges. Le *réseau d'essai* contient les services effectivement interrogés par les outils. Le contrôleur possède une interface de gestion et une interface vers le bridge de benchmark. L'accès distant documenté utilise Tailscale ; les règles de sortie installées séparent les besoins de gestion des échanges d'audit. Cette séparation

décrite par le provisionnement doit être vérifiée sur la cible avant une campagne : une configuration dans Git ne démontre pas son application effective [45, 47].

La figure 3 résume ces responsabilités. Le laboratoire personnel nommé `pve` est explicitement exclu du projet. Une panne de nato ne justifie aucun déplacement de scénario, de secret ou de configuration vers cette autre infrastructure. L'identité de la cible prime sur la présence d'un inventaire ou sur la possibilité technique d'établir une connexion [44].

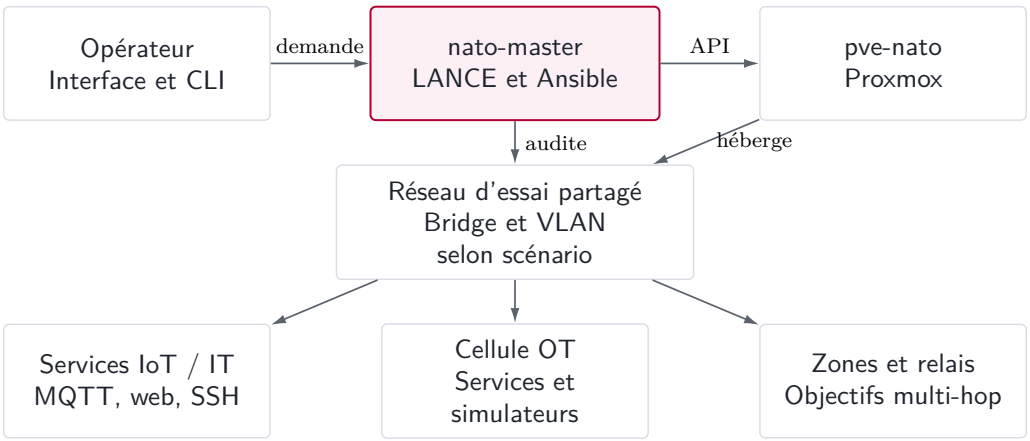


Figure 3 – Organisation logique de l'infrastructure implémentée. Les trois branches inférieures représentent des compositions possibles ; elles ne décrivent pas trois laboratoires disponibles simultanément. Le dessin ne représente ni un état déployé vérifié au 15 octobre 2026, ni les règles exactes de connectivité de chaque scénario.

4.2 Topologies, zones et signification d'un pivot

Le corpus couvre plusieurs organisations de réseau. Une topologie plate place les services dans une surface directement explorable ; une étoile organise les dépendances autour d'une passerelle ; les configurations de bâtiment, de ville connectée ou d'edge/cloud distribuent des rôles fonctionnels. Les configurations IT/IoT/OT introduisent des zones de confiance différentes. Leur intérêt est de faire varier l'information disponible, la diversité des services et les chemins nécessaires pour atteindre une ressource. Le nom d'une zone décrit une fonction expérimentale ; il ne suffit pas à établir un filtrage effectif [48, 49].

La segmentation doit être lue à plusieurs niveaux. Un VLAN sépare des domaines de communication sur le bridge ; une route indique comment joindre un autre sous-réseau ; un pare-feu peut autoriser ou interdire ce passage. Un hôte à plusieurs interfaces peut relier deux segments sans que tous les services soient accessibles depuis le point de départ. Le playbook de déploiement configure ces propriétés au moyen d'interfaces étiquetées, d'adresses secondaires et, pour certains rôles, de l'absence de passerelle. S13 possède une configuration particulière de bridge conscient des VLAN et de routes vers les zones. Les scénarios de pivots récents emploient aussi des interfaces temporaires de préparation : le contrôle de leur retrait et de l'absence d'accès direct relève de l'injection et de la vérification [46, 51].

S20 fournit un exemple plus précis qu'une simple étiquette « réseau segmenté ». Sa topologie définit un point d'entrée, un relais à plusieurs interfaces et une ressource finale dans un second segment isolé. Les liens attendus imposent un accès SSH vers le premier hôte, puis un accès SSH vers le relais, enfin un accès HTTP vers la ressource. Les variantes S24–S27 définissent des branches, des chemins asymétriques ou une cascade plus profonde. Ce sont des objectifs du support expérimental, dont les playbooks comportent des contrôles de chemin direct ; leur présence ne démontre pas que LANCE les a parcourus [50, 51].

Un *pivot réseau* exige une action depuis un accès précédemment obtenu vers une cible devenue joignable grâce à cet accès. Récupérer un secret sur un site puis l'employer sur un autre service directement accessible constitue une dépendance logique, sans prouver un changement de position réseau. De même, plusieurs opérations d'API ne créent pas plusieurs sauts. Cette distinction est décisive pour le projet : le pipeline courant ne produit pas la provenance causale nécessaire pour mesurer ses transitions réseau. Les noms « pivot » ou « multi-hop » ne doivent donc pas être convertis en résultats de compromission [49].

4.3 Familles de services et degré de réalisme

La messagerie MQTT suit un modèle de publication et de souscription : un client publie dans un canal nommé *topic* et d'autres clients s'y abonnent. Les ACL sont les règles qui autorisent ces opérations. TLS protège le transport ; mTLS ajoute une authentification mutuelle par certificats. OTA désigne une mise à jour distante. Ces propriétés sont séparées : chiffrer un échange ne garantit pas que le client possède le droit de lire ou de modifier la ressource.

Le laboratoire représente l'IoT par ses dépendances réseau et applicatives : messagerie entre équipements, interfaces d'administration, bases de données, passerelles, vidéo et systèmes industriels. Certains composants sont des services courants configurés pour le benchmark ; d'autres sont des simulateurs déterministes conçus pour isoler une propriété. Cette diversité est utile, mais elle impose de décrire la fidélité de chaque expérience. Le tableau 3 donne les principales familles sans leur attribuer une couverture expérimentale globale [49, 51].

Famille	Services représentés	Contrôles pertinents	Limites du support
Messagerie IoT	MQTT/Mosquitto ; CoAP dans certaines compositions	Connexion, lecture d'un sujet, publication et authentification, vérifiées séparément	Une opération admise ne décrit pas toutes les ACL ; aucun réseau radio réel n'en découle.
Administration et données	SSH, HTTP, FTP, SNMP, MariaDB, Redis, Node-RED	Authentification, accès à une ressource, exposition de configuration et refus attendu	Les configurations de laboratoire ne représentent pas toutes les versions ou installations d'un produit.
Vidéo et bâtiment	Façades caméra HTTP, rôle NVR, interfaces de bâtiment	Accès à l'administration et aux configurations exposées	Pas de flux RTSP réel, de conformité ONVIF ni de firmware constructeur démontrés par ces façades.
Identité et cloud	Enrôlement, certificats, mTLS, OTA, métadonnées cloud	Identité, révocation, confiance, version et autorisation	Cycles de vie et cloud simulés ; pas de validation d'un fournisseur cloud ou d'une puce sécurisée.
OT	Modbus/TCP ; simulateurs OPC UA et BACnet	Écriture, lecture et restauration d'un point de test ; refus du profil durci	Aucun automate physique ni effet sur un procédé industriel réel.

Table 3 – Lecture comparative des surfaces réseau présentes dans le dépôt. Les contrôles sont des objectifs ou mécanismes de vérification disponibles ; le tableau ne rapporte pas une campagne réalisée sur toutes ces familles.

S19 illustre l'intérêt d'un contrôle borné : les services exposent des valeurs de processus jetables, et la vérification Modbus effectue un cycle lecture–écriture–lecture–restauration. Des profils durcis OPC UA et BACnet doivent refuser les écritures anonymes. On peut ainsi étudier l'autorisation sans commander un équipement industriel. Toutefois, les échanges et points simulés ne couvrent pas l'ensemble des piles protocolaires : la négociation, l'identité et les opérations proposées restent celles du simulateur [52, 51].

La même prudence s'applique aux autres appellations. S16 combine des certificats OpenSSL réels avec un cycle d'identité simulé ; cela n'établit aucune extraction physique de clé. La découverte UDP de S23 ne reproduit pas une liaison Wi-Fi, LoRaWAN, Zigbee ou BLE. La « réplique NATO Lab » représente des rôles logiciels, sans attester la reproduction de tout le matériel historique. Ce périmètre permet des essais répétables sur IP ; les expériences radio, matérielles et sur produits réels constituent des travaux distincts [49].

4.4 Composition, posture et vérité terrain

Le catalogue versionné **3.2.0** identifie 29 scénarios publics : S1–S19 pour le développement et S20–S29 pour un test public provisoire. Ce catalogue est un registre d'identifiants et de groupes. Les machines et liens résident dans les topologies ; les propriétés de sécurité résident dans les packs ; la définition de scénario sélectionne ces éléments et une posture. La vérité terrain, ou *ground truth*, fournit à l'évaluateur les propriétés et objectifs attendus. Les paramètres Ansible traduisent cette composition vers le laboratoire. Ces couches doivent rester cohérentes : ajouter un YAML ne crée pas automatiquement un scénario déployable [53, 48].

Une composition peut associer une topologie contenant une passerelle et un service de messagerie, un pack décrivant la propriété d'accès anonyme à examiner et une posture autorisant ou refusant cet accès. La topologie précise les équipements et liens ; le pack fixe la propriété étudiée ; la posture fixe la configuration qui doit l'exposer ou la bloquer. L'infrastructure installe cette configuration et la vérité terrain indique les résultats attendus à l'évaluateur. Ces objets doivent correspondre : un évaluateur correct donnerait un score trompeur si le service déployé ne présentait pas la propriété annoncée.

La posture évite de réduire le benchmark à une collection de services tous vulnérables. Des profils vulnérables, durcis et proches d'un cas vulnérable permettent de varier la prévalence des défauts et d'introduire des contrôles négatifs. S14 et S21 ciblent notamment la précision en présence de services majoritairement protégés. Dans S19, une découverte autorisée peut coexister avec une écriture refusée : la simple présence d'un port ou d'une réponse ne doit pas être évaluée comme un défaut d'autorisation [48, 52].

La vérité terrain appartient à l'évaluateur. La topologie publique peut être fournie au pipeline ; le mode aveugle demande une découverte active. Donner les réponses attendues à l'agent invaliderait l'interprétation de sa découverte. La séparation des groupes a également une limite explicite : l'indépendance historique du test public vis-à-vis des choix de développement n'a pas été établie. Une comparaison de généralisation demanderait donc un gel du système et un jeu réellement indépendant [48].

Le parcours manuel et Scenario Lab étendent le laboratoire au-delà du catalogue. Ils produisent des bundles de scénario, topologie, vérité terrain et plans d'exécution. Les variantes exportées réservent une plage d'identifiants Proxmox et écrivent une surcouche de variables, sans modifier le catalogue officiel. La composition avancée peut aussi rester en mode de prévisualisation lorsqu'un adaptateur d'exécution manque. Les résultats de ces parcours doivent conserver leur origine : ils ne s'ajoutent pas silencieusement aux scores S1–S29 [54, 55].

4.5 Préparation, réservation, contrôle et remise en état

Une exécution expérimentale commence avant le premier appel au modèle. Le contrôleur prépare les artefacts et, si le verrou commun est activé, réserve le laboratoire. Le verrou interprocessus couvre le pipeline, le déploiement et le nettoyage ; il est partagé entre les instances applicatives. L'attente est visible et peut être annulée pour un pipeline. Cette réservation est coopérative, sans ordre d'arrivée (FIFO) garanti. Un playbook lancé directement, une ancienne version ou une intervention Proxmox extérieure peuvent la contourner [58, 59].

Le cycle automatique appelle successivement le déploiement des charges `03_deploy_scenario.yml`, l'injection des propriétés `04_inject_vulns.yml`, puis la vérification `06_verify.yml`. Le déploiement contrôle les modèles nécessaires, l'appartenance des charges existantes et les conflits de scénarios ; il configure ensuite les interfaces et attend la disponibilité des services. L'injection prépare les configurations et simulateurs. Le peuplement `05_populate_services.yml` existe comme étape distincte et intervient notamment dans le reset ; il ne faut pas le supposer systématiquement appelé par le cycle automatique courant [56, 46, 57].

La vérification préalable, ou *preflight*, cherche à détecter un laboratoire incorrect avant de consommer des ressources d'audit. Elle comprend des contrôles de configuration et de comportement : accès MQTT effectif, ressource HTTP attendue, écriture OT bornée, contrôles négatifs et absence de chemin direct pour certains scénarios. Un échec arrête la préparation et déclenche un nettoyage. Son contrat annonce néanmoins explicitement que toutes les propriétés de vérité terrain ne sont pas certifiées. Son succès est une condition de préparation, jamais une mesure du rappel ou de la compétence de l'agent [56, 51].

Après l'essai, le nettoyage retire les charges du scénario ; les variantes générées libèrent leur réservation locale après succès du playbook. Un mode « déployer seulement » peut conserver le scénario, mais ne réserve pas le laboratoire indéfiniment. Le prochain run peut le remplacer. Le reset redémarre les services puis réinjecte et repeuple leurs données : il évite une reconstruction complète, sans constituer à lui seul une preuve d'identité de tous les états antérieurs. Pour une comparaison, il faut enregistrer la procédure retenue et ses contrôles plutôt que supposer deux essais équivalents [56, 57].

4.6 Intégration continue, exploitation et avancement du volet

L'exploitation actuelle prévoit trois emplacements sur nato : l'instance stable sur `main`, puis deux instances associées à `dev/1` et `dev/2`. Elles possèdent des environnements Python, bases de données et artefacts distincts, tout en partageant le réseau et les charges du laboratoire. Une activité applicative commune indique les opérations déclarées et l'attente du verrou. Elle ne constitue pas un inventaire des actions CLI ou Proxmox et ne permet pas d'inférer la disponibilité physique à partir d'une interface inactive [58].

Le workflow d'intégrité valide les changements avant d'appeler le workflow de déploiement sur le runner `nato-master`. Le parcours complet contrôle les compositions, la syntaxe Ansible, les tests applicatifs et d'entraînement, puis l'absence de vérité terrain dans l'image du worker isolé. Les modifications limitées aux documents reconnus suivent un parcours léger, sans déploiement. Les pull requests valident sans installer leur code sur le laboratoire. Le script de déploiement vérifie l'identité attendue du runner, acquiert le verrou, contrôle l'activité, met à jour l'emplacement et vérifie le commit annoncé après redémarrage. Ces mécanismes décrivent le circuit de déploiement implémenté [60, 61, 58].

Acquis. LANCE dispose d'une infrastructure décrite et automatisée, de compositions réseau variées, d'un cycle de préparation contrôlé et d'une coordination entre plusieurs instances. Les propriétés négatives et les conditions de connectivité sont traitées comme des objets vérifiables. **Limites.** La disponibilité actuelle, l'installation effective des trois instances et le déploiement réussi des 29 scénarios ne sont pas établis ici. La virtualisation, les simulateurs, les modèles de base, les dépendances de paquets et les ressources de Proxmox bornent la fidélité et la répétabilité. **Étapes suivantes.** Une campagne doit figer les révisions et contrats, consigner modèles système et versions de services, archiver preflight, routes, profils, remise en état et incidents, puis établir une matrice des scénarios réellement déployés et vérifiés. La preuve causale des pivots doit être instrumentée avant toute conclusion sur les chaînes réseau.

5 Sécurité, confiance et construction de la preuve

5.1 Ce que LANCE cherche à établir et à protéger

La fonction de sécurité de LANCE comporte deux responsabilités : établir des constats sur un réseau autorisé et maîtriser les actions de l'instrument qui les produit. La première concerne les équipements audités : authentification, confidentialité des données, intégrité des mises à jour, droits applicatifs et exposition des services. La seconde concerne le moteur d'audit, ses accès au laboratoire, les fournisseurs de modèles et les artefacts contenant les observations. Une conclusion correcte n'excuse pas une action hors périmètre ; une exécution correctement bornée ne garantit pas une conclusion exacte.

Les opérations de laboratoire ciblent nato/pve-nato et son contrôleur nato-master ; le homelab personnel pve reste hors périmètre. Dans le moteur, les sous-réseaux du scénario délimitent les destinations admises : l'exécuteur confronte les adresses demandées à ces plages avant l'action. Cette frontière est une garde applicative dont l'efficacité dépend aussi du comportement des outils et de la configuration réseau effective [32].

Le modèle de confiance se lit en trois niveaux. Le modèle de langage formule des hypothèses et choisit des actions ; son texte reste une déclaration. Les outils produisent des observations dont l'exécuteur conserve les arguments et les résultats. Enfin, des règles de preuve et l'évaluateur déterminent ce que ces observations soutiennent. La vérité terrain du benchmark sert à mesurer la correspondance avec les propriétés attendues ; elle ne doit pas devenir une information offerte à l'agent. Cette organisation réduit la confiance accordée à la narration, mais conserve des dépendances : exactitude des adaptateurs, authenticité des réponses du laboratoire et pertinence de la référence. Une réponse simulée reste une preuve relative à cette simulation.

5.2 De l'hypothèse à une observation attribuable

Un constat doit décrire une propriété suffisamment précise pour être vérifiable. « MQTT est ouvert » et « des données sensibles sont lisibles sans authentification sur un topic donné » n'ont ni le même sens ni les mêmes conditions de confirmation. L'identité conservatrice d'un constat comprend notamment la cible, le type, le service, le port, le transport, la ressource, les informations de produit/version et les conditions annoncées. Les identifiants de constats et les références d'outils servent à la traçabilité, sans rendre deux hypothèses sémantiquement équivalentes [62].

La frontière commune d'exécution attribue à chaque appel une référence `tc-...`, une phase, une origine `runner`, une séquence, les arguments effectifs, le résultat et des informations temporelles. Elle conserve séparément le contexte de l'hypothèse. Ainsi, un candidat annonçant le port 9001 ne transforme pas en preuve sur 9001 une action réellement exécutée sur 9002. Si l'archivage échoue, le moteur marque la perte d'intégrité et bloque la poursuite normale : une action non archivable rend le run non vérifiable [32].

L'évaluateur relit ce journal au lieu d'accepter le seul livrable final. Il rapproche l'outil, la cible issue des arguments, le port, la ressource et l'identifiant de constat. Une référence explicitement citée ne dispense pas de ces vérifications. Une trace attribuable à plusieurs constats n'est pas distribuée arbitrairement ; les références dupliquées sont exclues du rapprochement. Ces contrôles répondent à un risque concret : réutiliser une réponse réelle pour soutenir une autre machine, un autre chemin ou une autre propriété [63].

La figure 4 résume cette chaîne. Son journal constitue une provenance applicative, pas une attestation cryptographique indépendante de l'hôte qui l'écrit. La traçabilité facilite une revue contradictoire ; elle ne démontre pas, à elle seule, la vérité du contenu reçu.

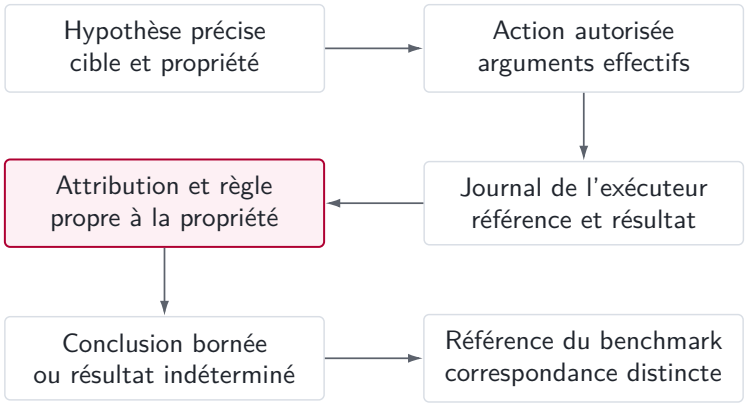


Figure 4 – Chaîne de preuve implémentée, représentée schématiquement. Le dernier rapprochement mesure l'accord avec la référence ; il reste distinct de l'admission sémantique d'une observation. Aucune flèche ne représente une validation nouvelle du laboratoire.

5.3 Des critères adaptés aux protocoles et aux propriétés

La vérification exige une observation fraîche et pertinente pour l'hypothèse. Des plans par famille guident le choix d'outil et la condition de succès ; le profil complet conserve son choix autonome d'actions. L'interprétation partagée des traces évite que les profils complet et compact appliquent deux sens différents au même résultat [64, 65]. Le tableau 4 donne des exemples de portée probatoire.

Propriété annoncée	Observation insuffisante	Preuve pertinente et portée
Faiblesse SSH	Port ouvert, conseil de durcissement ou nom de CVE isolé	Algorithme explicitement observé et visé par la politique ; pour un accès, authentification et résultat d'exécution attribués.
Lecture MQTT anonyme	Connexion TCP, upgrade WebSocket ou texte mentionnant un topic	Échange applicatif accepté et message attribué au topic ; une fuite demande un contenu sensible effectivement reçu.
Accès HTTP sans authentification	HTTP 200 d'une page de connexion ou réponse obtenue avec cookie	Contenu privilégié pertinent reçu sans credentials, sur la destination et le chemin revendiqués.
Intégrité OTA	Présence d'une route de mise à jour ou fichier signé	Acceptation observée d'une requête testant la propriété ; toute conclusion reste limitée au mécanisme et au simulateur concernés.
Identité partagée	Deux requêtes identiques vers le même équipement	Observations valides sur des équipements distincts ; aucune déduction automatique de possession d'une clé privée.

Table 4 – Exemples conceptuels issus des règles locales. Le tableau ne décrit pas une campagne réalisée ni un catalogue exhaustif de garanties.

Pour SSH, la politique cryptographique finie examine les lignes d'algorithmes effectivement offertes par le serveur. Une recommandation ou un avertissement en prose ne suffit pas. Une violation de cette politique décrit une configuration ; elle ne prouve ni négociation effective de l'algorithme ni exploitation. Une authentification par une paire connue comme faible ne démontre pas davantage qu'il s'agit du mot de passe d'usine du constructeur [66, 65].

Pour MQTT, les messages structurés lient le topic réel au filtre de souscription et au topic revendiqué. Le format ancien ne permet pas d'élargir librement cette attribution. MQTT sur

WebSocket exige son échange applicatif : réponses de connexion et de souscription acceptées, puis message reçu sur le chemin et le topic attendus. Le statut HTTP 101 décrit seulement le transport WebSocket. Pour HTTP, une redirection ne transfère pas la preuve de la destination finale vers l'URL initiale [67, 68].

Les cas OTA et identité montrent enfin la nécessité de plusieurs observations. Le scanner reconnaît des contrats bornés de mises à jour signées : version obsolète, métadonnées non couvertes ou périmètre de clé. Les contrôles d'identité distinguent des machines différentes et rejettent les duplications de requêtes ou d'alias. Ces extracteurs propres aux scénarios ne constituent pas un vérificateur universel de chaîne de confiance, de révocation ou de firmware industriel [69, 78].

5.4 Confirmation, résultat négatif et résultat indéterminé

Le point méthodologique central est simple : `success:true` exprime le succès déclaré d'une opération selon son adaptateur. Il ne dit pas quelle propriété de sécurité est établie. Un client HTTP peut terminer normalement après une réponse interdite ; une requête peut réussir vers la mauvaise ressource ; un service peut répondre sans exposer de donnée sensible. La preuve doit être interprétée avec la requête et les critères du constat. De même, un rapport bien formé démontre une conformité de format, pas une vulnérabilité [65, 68].

Les preuves directes portent sur l'objet annoncé : contenu effectivement obtenu, réponse protocolaire corrélée ou résultat d'exécution. Les indices indirects, tels qu'une bannière, un nom de fichier ou une ressemblance de version, peuvent justifier une hypothèse ou un constat plus étroit. Ils n'autorisent pas à promouvoir une possibilité en accès démontré. La lecture de noms dans un listing FTP ne prouve pas le contenu des fichiers ; un `robots.txt` ne démontre ni leur existence ni leur accessibilité.

La politique de restitution distingue les états confirmés, non concluants, en erreur et non testés. Seuls les constats confirmés avec un niveau de preuve suffisant entrent dans les tableaux de déclarations soutenues du rapport. Les fiches de toutes les hypothèses, y compris celles sans test ou non concluantes, restent présentes dans `06_report.md` ; les résultats structurés et `tool_calls.jsonl` conservent les détails et les traces. L'évaluateur contrôle ensuite indépendamment l'attribution et la sémantique. Une preuve acceptée peut encore être hors référence : l'accord avec la vérité terrain demeure une autre question [70, 27, 68].

Un refus d'authentification fournit un contrôle négatif pour cette requête. Des clés publiques distinctes constituent un contrôle négatif pour l'hypothèse de partage considérée. En revanche, timeout, outil indisponible ou tentative échouée ne prouvent pas l'absence générale de faille. Aucun validateur générique de preuve d'absence n'est implémenté. L'état `refuted` reste réservé ; `FAILED` et `NOT_EXPLOITABLE` restent non concluants. Cette prudence évite de déclarer un équipement sûr simplement parce que l'audit n'a pas réussi à le tester.

Considérons une hypothèse de lecture HTTP sans authentification. Si la ressource sensible est obtenue sans identité, le constat peut être confirmé dans ce périmètre. Si le serveur renvoie un refus explicite, cette requête documente un contrôle négatif ; elle ne décrit pas tous les chemins du serveur. Si le client expire ou reçoit une réponse impossible à interpréter, le résultat reste indéterminé. L'opérateur doit retrouver ensemble la propriété testée, la requête, la réponse et le verdict. Le code de sortie du client ne permet pas, à lui seul, de choisir entre ces conclusions.

5.5 Observation de service, accès obtenu et pivot réseau

La phase d'intrusion traite trois événements différents : contacter un service, obtenir un accès et agir depuis cet accès vers une autre machine. Un HTTP 200 ordinaire ou une souscription MQTT ne démontre pas un contrôle de la machine. Les règles d'accès reconnaissent des formes bornées d'authentification ou d'exécution. Pour SSH, elles demandent une requête identifiable et un résultat

compatible avec une commande exécutée ; pour l'authentification HTTP, un contrôle comparatif doit distinguer la réponse sans credentials de la réponse authentifiée [71].

La projection destinée à l'API et à l'interface dérive les accès corroborés des traces de phase 5 portant une origine exécuteur et une référence unique. Le fichier écrit par le modèle contribue seulement à des compteurs de déclarations. Un journal absent, mal formé, ambigu ou incompatible ne devient pas une liste d'accès vide interprétée comme un résultat expérimental négatif : la projection peut être indisponible, avec un motif explicite [72].

La limite actuelle est structurante : aucune provenance causale de transition réseau n'est produite. La projection expose donc des transitions vides et `transition_evidence_available=false`. Deux connexions réussies depuis l'auditeur ne prouvent pas qu'une machine a servi de relais vers l'autre. Les taux de pivot restent indisponibles, plutôt que remplacés par zéro. Un chemin direct peut être évalué sans saut ; les chemins multi-sauts demandent une preuve supplémentaire [68].

Enfin, la condition anti-inactivité du profil complet bloque la clôture sans tentative observable lorsque des points d'entrée existent. Elle améliore la réalité des actions enregistrées ; elle ne transforme ni tentative ni erreur en compromission. La progression vers une mesure des pivots reste un travail à concevoir et à instrumenter [73].

5.6 Sécurité du moteur et de l'application de pilotage

Le contrôle commun des outils vérifie les destinations par rapport aux CIDR, c'est-à-dire aux plages réseau déclarées. Il refuse les adresses hors périmètre, les noms dont l'appartenance n'est pas vérifiable et certaines commandes réseau lancées depuis un hôte compromis. L'exécution Python libre est refusée faute de confinement vérifiable. Les outils structurés et la désactivation des redirections HTTP automatiques réduisent les possibilités de contourner la cible annoncée [32, 74].

Ce sont des gardes applicatives, pas un bac à sable du système d'exploitation. Le parseur ne prouve pas le comportement de tout programme arbitraire ni l'absence de destination dérivée. Les budgets, délais et arrêts coopératifs bornent l'activité sans prouver l'innocuité de chaque opération. Des sondes d'écriture, de mise à jour ou de changement de droits peuvent modifier l'état d'un simulateur : leur autorisation et la remise en état du scénario restent distinctes du verdict de sécurité.

L'application protège les mutations administratives par une clé serveur ou une session révocable. Une clé absente bloque ces mutations avant le traitement ; la comparaison de clé utilise une primitive en temps constant. Les sessions durent huit heures, résident en mémoire et utilisent un cookie HttpOnly/SameSite strict. Les écritures par cookie demandent un en-tête spécifique et contrôlent l'origine. Le cookie devient Secure en HTTPS [75, 76].

Le périmètre de cette protection doit être décrit exactement : les routes de démarrage d'un run et d'un batch sont publiques dans le contrat courant. L'absence d'origine CORS autorisée ne remplace pas l'authentification d'un client réseau. Le guide réserve donc cet accès à un réseau de confiance ou à un tunnel chiffré. Le stockage des sessions dans un processus limite aussi l'extension à plusieurs workers. Enfin, les journaux contiennent arguments et résultats ; ils doivent être traités comme des artefacts potentiellement sensibles, et non comme des données publiques par défaut.

5.7 Acquis, limites et prochaines étapes de sécurité

Les acquis implémentés sont la séparation entre déclaration et observation, la référence d'exécution, l'attribution structurelle des preuves, des règles par protocole et une projection conservatrice des accès. Le dépôt contient des tests ciblés : refus d'une destination WebSocket hors périmètre, non-admission d'un upgrade comme preuve MQTT, cohérence des accès entre profils, refus de mutations administratives et contrôles d'identité sur des machines distinctes. Leur lecture montre les propriétés visées ; aucune réexécution de ces tests n'a été réalisée pour cette rédaction [77, 78, 79].

Les limites restent substantielles : critères finis, marqueurs textuels pour certaines réponses HTTP, extracteurs liés aux simulateurs, dépendance envers l'intégrité de l'exécuteur et absence de transitions causales. Un scénario non couvert ou une réponse inhabituelle peut rester indéterminé. Une suite de tests correcte ne démontre ni la sécurité globale du moteur ni la fidélité complète du laboratoire.

Les contrats actuels sont `strict-v3.16` et `evidence-v15`. Les corrections successives ont resserré notamment les preuves SSH, MQTT, WebSocket et les lectures natives FTP/Redis/SNMP. Les scores historiques conservent leur contrat d'origine ; les réinterpréter avec une nouvelle règle ne constitue pas une nouvelle expérience [80, 68].

Les prochaines étapes proposées sont de formaliser davantage les sorties structurées des vérificateurs, documenter les propriétés exactes de chaque sonde, renforcer les contrôles positifs et négatifs lors de la préparation du laboratoire, puis produire une provenance d'exécution depuis un accès pour mesurer réellement les pivots. L'isolation des exécutants et le contrôle d'accès au pilotage doivent évoluer avec le contexte de déploiement. Ces objectifs doivent être validés séparément : conformité du logiciel, réalité des propriétés du laboratoire et qualité des résultats d'audit.

6 Application, expérience opérateur et exploitation

6.1 Transformer le moteur d'audit en un parcours utilisable

L'application donne une continuité aux opérations qui entourent un audit : choisir un périmètre, vérifier une configuration, lancer un traitement, suivre son déroulement, examiner les résultats et conserver les éléments permettant de les relire. Ce rôle dépasse l'affichage d'une réponse du modèle. Une exécution peut produire des détections utiles tout en échouant pendant la vérification, la rédaction ou le nettoyage ; l'opérateur doit pouvoir comprendre ces situations sans reconstruire manuellement l'ensemble des journaux. Le serveur FastAPI, le tableau de bord et les artefacts par exécution forment aujourd'hui cette chaîne de restitution [81, 82, 83].

Trois responsabilités humaines se distinguent, sans constituer un système complet de rôles applicatifs. L'opérateur prépare et suit un audit autorisé. L'expérimentateur sélectionne des configurations et interprète plusieurs exécutions dans le cadre d'une comparaison. L'administrateur configure les fournisseurs et intervient sur les opérations protégées. Le lecteur d'un rapport peut enfin consulter les résultats sans être celui qui a lancé le traitement. Cette distinction décrit les usages ; elle ne doit pas être confondue avec une gestion d'utilisateurs, de projets et de permissions individualisées.

Dans un parcours courant, l'opérateur sélectionne le moteur puis, pour LANCE, un scénario et un couple fournisseur–modèle. Il précise le profil d'exécution, les options applicables et le nettoyage automatique. Le tableau de bord présente ensuite la préparation, les phases et leur progression. Après la fin, l'historique permet de rouvrir le résumé, les observations et les fichiers du run. L'exemple conceptuel d'un service SSH détecté illustre ce parcours : voir le service dans la topologie oriente la lecture, mais seul le résultat de vérification associé à ses traces permet d'interpréter une confirmation. Le parcours reste valable si le contrôle est négatif ou incomplet : le produit doit restituer ces issues avec leur sens propre [84, 83].

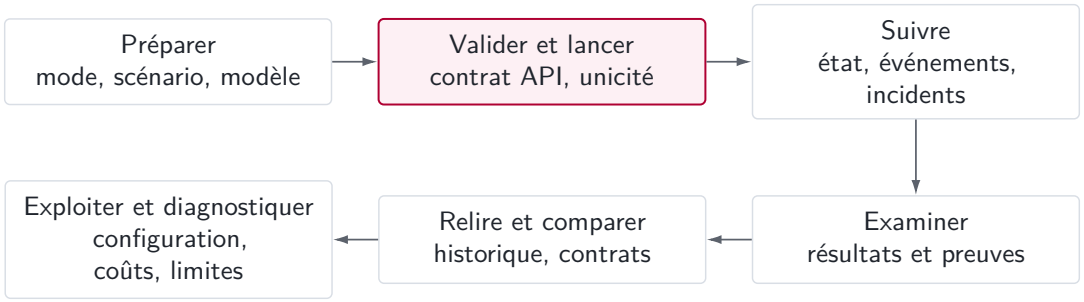


Figure 5 – Parcours applicatif conceptuel. La progression des écrans ne préjuge ni de la réussite de l'audit, ni de la présence d'une preuve. Les opérations administratives conservent leur authentification propre.

6.2 Des modes et profils qui répondent à des besoins différents

Le choix initial détermine ce qui sera exécuté et ce que les résultats pourront signifier. `lance` expose le pipeline complet. `rules` propose l'analyse et la vérification déterministes d'un scénario. `vanilla` expose une boucle modèle–outils bornée sur un réseau explicitement déclaré. `scripted` exécute une procédure versionnée. Ces choix partagent une surface de lancement et de suivi, mais leurs résultats ne sont pas interchangeables [84, 85].

Table 5 – Modes disponibles et portée de leur restitution applicative.

Mode	Parcours	Données visibles	Limites de lecture
`lance`	Scénario, personnalisé ou batch ; modèle requis pour l'audit	Phases, topologie, résultats, rapport et évaluation applicable	Un rapport présent ne suffit pas à établir la réussite.
`rules`	Scénario ou architecture personnalisée ; sans modèle	Détections et vérifications par équipement, traces, résumé déterministe	Sans scoring benchmark dans ce parcours ; pas de batch applicatif.
`vanilla`	CIDR explicite, fournisseur et modèle	Appels bornés, résumé, statut, consommation	Sans déploiement de scénario ni score benchmark.
`scripted`	`smoke-v1` hors ligne ou `discovery-v1`	Étapes, observations disponibles, résumé et statut	La démonstration hors ligne ne constitue pas un audit réseau.

Les profils `compact` et `full` règlent l'orchestration de LANCE. Le profil compact réduit les budgets et sélectionne des outils adaptés ; le profil complet conserve une surface et des budgets plus étendus. Avec `auto`, la priorité va au choix du run, puis au réglage du modèle, puis à ses paramètres actifs ou totaux connus : jusqu'à 32 milliards, le profil est compact ; sans métadonnées, il est complet. La résolution est enregistrée. Ce seuil organise les ressources ; il ne mesure ni la compétence du modèle ni la gravité des constats [33].

Le parcours personnalisé sélectionne une architecture, une posture et des packs au lieu d'une simple entrée prédéfinie : il compose un scénario de laboratoire et en déclenche la préparation lorsque le déploiement est activé. Un réseau existant se déclare au pipeline par une plage d'adresses cible, sans identifiant de scénario ; le parcours de découverte enrichit alors le graphe sans créer de machines de scénario. Le mode aveugle d'un scénario cache sa topologie à l'agent tout en conservant son déploiement. Enfin, `vanilla` reste une boucle de reconnaissance distincte du pipeline à six phases [9, 84, 85].

Un batch, ou lot, LANCE enchaîne des scénarios séquentiellement et conserve l'identité de chaque tentative. Son bilan distingue les statuts terminaux, les incidents de nettoyage et les coûts

inconnus. Une tentative sans évaluation ne disparaît donc pas automatiquement du décompte. Les scénarios scellés relèvent du contrôleur spécialisé ; l'interface ordinaire ne devient pas un accès général à leur contenu [85, 92].

6.3 Configuration des modèles et préconditions du lancement

Le registre SQLite sépare le fournisseur du modèle. Le premier définit notamment le point d'accès (endpoint), c'est-à-dire l'adresse du service à contacter, et le nom de la variable d'environnement portant son secret ; la seconde entrée porte un identifiant, des prix éventuels et une politique de profil. Cette séparation évite de déduire un fournisseur du seul nom commercial d'un modèle. Pour un audit avec modèle, le lancement exige un fournisseur et un modèle explicites. Le déploiement seul et les moteurs déterministes répondent à d'autres contrats [12, 86, 85].

Le gestionnaire permet d'administrer des endpoints personnalisés, notamment locaux. Les anciens fournisseurs OpenRouter, Codex et Anthropic sont refusés comme moyens d'exécution dans l'état courant documenté ; leurs anciennes lignes et leurs résultats historiques peuvent rester lisibles. Le catalogue tarifaire OpenRouter a une fonction distincte d'estimation et ne reçoit pas les prompts par ce mécanisme. Il faut donc distinguer une entrée historique consultable, un modèle proposé au lancement et un endpoint effectivement joignable [87].

Lorsque l'option séquentielle est activée, les appels partageant la même origine de fournisseur attendent un accès commun avant d'envoyer leur requête. Ce séquencement évite d'imposer au serveur plusieurs générations simultanées ; les outils n'occupent pas ce verrou. L'application sépare l'attente d'admission du temps de requête. L'opérateur distingue ainsi un fournisseur qui traite lentement une demande d'un appel qui attend encore son tour [40, 88].

Avant un run, l'opérateur doit ainsi disposer d'une configuration cohérente, d'identifiants valides lorsque requis, d'un endpoint disponible et des dépendances correspondant au mode. La présence d'un modèle dans le registre ne vérifie aucune de ces disponibilités en direct. L'API valide les combinaisons de paramètres ; elle ne remplace ni la préparation de l'environnement ni l'autorisation d'auditer la cible.

6.4 Coordination, événements et continuité du suivi

Une instance applicative est un processus serveur LANCE avec son état d'exécution. Elle conserve un seul run actif. Le coordinateur centralise l'état courant, le signal d'arrêt coopératif, la progression et la diffusion des événements ; le runner reste responsable de son exécution. Une demande en conflit avec un run actif ou un nettoyage en cours est refusée. La réponse de lancement confirme l'admission de la demande, sans annoncer déjà la réussite du traitement [89, 85].

Le navigateur combine un instantané de statut avec un flux SSE (*Server-Sent Events*). Ce flux transporte les débuts et fins de phases, l'avancement par équipement, les opérations de préparation et les incidents. Chaque lecteur possède son curseur : ouvrir un second navigateur ne retire pas les événements du premier. Le journal conserve une fenêtre bornée de 1 000 événements ; lorsqu'une absence dépasse cette fenêtre, le lecteur reçoit un avertissement et doit consulter les journaux complets du run. Cette rétention en mémoire permet de reprendre un suivi bref, sans constituer une archive durable de l'exécution [90].

Le coordinateur conserve aussi les événements récents utiles à la restitution de l'état après rechargement. L'état d'attente du laboratoire est distinct d'une phase qui travaille : sans cette distinction, une attente légitime pourrait être interprétée comme un blocage du modèle. Une demande d'arrêt place le run dans l'état d'arrêt en cours ; elle n'implique pas une interruption instantanée de chaque opération déjà engagée [89].

Une vue d'activité peut réunir les trois instances cohébergées connues, via des pairs locaux fixes. Elle annonce une disponibilité partielle lorsqu'un pair ne répond pas, au lieu de le déclarer libre. Cette vue aide l'utilisateur à comprendre une ressource partagée ; elle ne remplace pas la

réservation effective du laboratoire. Les accès et la disponibilité réels des instances n'ont pas été vérifiés pour ce bilan documentaire [91].

6.5 Topologie et résultats : montrer ce que les données permettent

Le graphe interactif permet de situer les équipements, leurs liens et leurs observations ; un clic ouvre la fiche du nœud. Pour l'audit par règles, la projection par équipement distingue détections, confirmations, contrôles en attente, erreurs de scan et erreurs de vérification. Un nœud sans adresse IP reste visible avec la mention de non-ciblage. Cette représentation évite d'assimiler un équipement non contrôlé à un équipement déclaré sain [83, 94].

La restitution d'intrusion applique une séparation analogue. Le navigateur qualifie un accès uniquement depuis une projection serveur issue du journal d'observations, avec des références de preuve non vides. Une déclaration brute du modèle ne colore pas à elle seule un équipement comme compromis. Si cette projection est absente ou incompatible, l'accès est indiqué comme indisponible. Deux accès corroborés ne créent pas un lien de pivot : les transitions réseau causales ne sont pas fournies par l'exécuteur actuel et l'interface ne les reconstitue pas à partir d'une simple succession d'accès [83, 95].

Cette prudence est essentielle pour un rapport visuel. Un lecteur peut comprendre une couleur ou une flèche avant de lire sa légende. Le graphe reste donc une projection d'informations ayant des statuts différents : structure attendue, observation et preuve d'accès doivent conserver leur origine. Les runs simples ouvrent leur propre résumé sans fabriquer une topologie LANCE ou une réussite d'exploitation. Pour les règles, la topologie publique sauvegardée avec le run permet de relire les résultats après nettoyage du scénario [84, 83].

6.6 Historique, comparaison et diagnostic des ressources

Chaque run dispose de son dossier et de métadonnées de cycle de vie. L'historique expose le mode, le statut, les fichiers et les informations disponibles sur la configuration. Les lecteurs peuvent ouvrir des livrables ou télécharger une archive. L'API ne déduit pas la réussite de la présence d'un rapport : une erreur d'intégrité probatoire, un nettoyage échoué ou une consommation incomplète peuvent modifier le statut présenté. Pour les anciens runs sans métadonnées suffisantes, la lecture reste conservatrice [82, 88].

La comparaison applicative place deux runs publics côte à côte et expose leurs données. La vue benchmark affiche aussi les versions des contrats de métrique et de preuve ainsi que leur compatibilité. Un score incompatible est marqué non comparable ; un résultat absent n'est pas transformé en zéro. Cela facilite la lecture de l'historique, mais le face-à-face graphique ne démontre pas que les deux essais partagent scénario, outils, droits, budget et conditions. Le contrôle d'une paire expérimentale demeure une exigence supplémentaire [83, 96].

La consommation constitue un autre axe de diagnostic. Les tables de runs et d'usage par phase conservent les tokens et les coûts connus ; les artefacts permettent de conserver la restitution avec l'exécution. Ces données décrivent l'usage du modèle selon les informations tarifaires disponibles, et non le coût complet de l'étude : préparation humaine, machines, entretien et relecture n'y sont pas automatiquement valorisés. Un total incomplet doit rester identifiable comme tel [12, 92].

Les diagnostics de fin distinguent notamment une connexion fournisseur impossible, un délai dépassé, une authentification refusée et une erreur d'exécution. Ils aident à expliquer pourquoi un audit final n'a pas été produit. Les résumés séparent également le cycle de vie, l'évaluation, la vérification et l'intrusion. Une exécution terminée peut donc être restituée avec incidents techniques sans que ses résultats intermédiaires soient effacés [82, 96].

6.7 Accès administratif, acquis et travail restant

Le contrat actuel laisse les lancements de runs et de batches accessibles sans clé ni session administrateur. Les autres mutations, dont l'arrêt, le nettoyage et les modifications de fournisseurs, exigent l'authentification administrative. Les lectures demeurent publiques dans l'application : le mécanisme ne rend pas les rapports et journaux confidentiels. L'environnement d'accès doit donc être défini en conséquence [87, 76].

Une session administrative révocable et conservée en mémoire dure huit heures. Son cookie est inaccessible au JavaScript du navigateur et limité aux requêtes du même site ; il porte l'attribut Secure sous HTTPS. Les mutations utilisant ce cookie demandent un en-tête spécifique et une origine admise. Le gestionnaire des fournisseurs conserve son champ de clé pour ces opérations. Ce mécanisme suppose une exploitation cohérente avec un processus serveur ; il ne constitue pas une administration multi-utilisateur complète [76, 87].

Le socle applicatif acquis est substantiel : sélection de moteurs, paramètres validés, suivi partagé entre lecteurs, relecture des artefacts, topologie liée au run et restitution explicite des réserves. Des tests ciblent les contrats de lancement, les états incomplets, les déclarations d'intrusion non probantes et les synthèses de fin. Leur lecture établit ce que la suite cherche à garantir ; elle ne constitue ni une nouvelle exécution de ces tests, ni une validation ergonomique auprès d'utilisateurs [93, 94, 95, 96].

L'étape suivante est de documenter et mesurer le travail humain nécessaire : temps de préparation, compréhension des incidents, corrections de configuration, relecture des preuves et décision finale. Une étude de tâches avec des auditeurs permettrait d'évaluer l'utilité du produit, l'accessibilité des informations et les erreurs d'interprétation. L'automatisation du parcours et la richesse de ses traces rendent cette mesure possible ; elles ne prouvent pas encore un gain de temps ou une autonomie opérationnelle générale.

7 Modèles, données et spécialisation par apprentissage

7.1 Pourquoi spécialiser un modèle local ?

Le volet apprentissage poursuit un objectif distinct de l'automatisation du réseau : rendre un modèle compact capable de respecter les contrats de LANCE. Reconnaître une vulnérabilité dans un texte ne suffit pas. Le modèle doit sélectionner un outil autorisé, produire ses arguments, interpréter une observation, conserver une preuve et enregistrer le livrable attendu. La spécialisation vise cette conduite opérationnelle, avec une base partagée pour limiter la mémoire nécessaire. L'hypothèse est qu'un apprentissage propre à chaque rôle améliore la conformité et l'efficacité ; ce gain doit encore être isolé expérimentalement des effets du prompt et des garde-fous logiciels. Le mémoire historique décrit cette motivation et la chaîne développée [97].

Trois mécanismes complémentaires interviennent. Le prompt fixe le rôle et les instructions de la requête. La recherche documentaire ajoute des connaissances au contexte sans modifier les poids : le magasin de connaissances expose des descriptions de vulnérabilités, des méthodes et un historique, avec stockage ChromaDB et plongements vectoriels configurés via Voyage AI. Enfin, l'ajustement supervisé modifie des paramètres entraînables pour apprendre une forme de comportement. Ces mécanismes ne se remplacent pas : une information retrouvée reste à vérifier sur la cible, et un adaptateur ne dispense pas de fournir les contrats actuels des outils [38, 37].

La couche fournisseur permet de consommer des endpoints compatibles OpenAI, locaux ou distants, selon le contrat de configuration de l'application. L'architecture n'impose donc pas les adaptateurs locaux à toutes les campagnes. Comparer un modèle distant au HMoE local doit conserver le même pipeline et expliciter les différences de budgets et de contexte [31].

7.2 QLoRA : une base quantifiée et des corrections spécialisées

Le parcours courant configure Qwen2.5-3B-Instruct, soit une famille de modèle d'environ trois milliards de paramètres. Les anciens profils Qwen2.5-0.5B restent conservés ; leur présence ne désigne pas la configuration active d'un service. Cette configuration 3B et les essais documentés avec Qwen3.8 :27b correspondent à des modèles différents : les résultats du second ne valident pas les adaptateurs du premier. QLoRA, pour *Quantized Low-Rank Adaptation*, associe une base quantifiée à des matrices de faible rang entraînables. Les poids de base restent gelés ; chaque expert apprend une correction. Pour une projection, le principe s'écrit

$$h = Q_4(W)x + \frac{\alpha}{r} B_e A_e x.$$

Dans cette expression, x est la représentation reçue par une couche et h sa sortie. W contient les poids fixes de la projection de base ; $Q_4(W)$ en conserve une approximation sur quatre bits. Les matrices A_e et B_e constituent l'adaptateur de l'expert e. Leur faible rang r réduit le nombre de paramètres à apprendre ; α règle l'amplitude de la correction. À l'entraînement, seule cette correction est ajustée. En service, une base commune est donc associée à la correction du rôle choisi. Quantification et partage visent une économie de mémoire ; son ampleur et son effet sur la qualité demandent une mesure reproductible [98, 99].

La configuration versionnée fixe une quantification NF4, *NormalFloat 4*, en quatre bits, la double quantification et le calcul en BF16, *Brain Floating Point* sur 16 bits. Les adaptateurs utilisent un rang de 16, un facteur $\alpha = 32$ et un abandon aléatoire de 0,05. Ils ciblent les quatre projections de l'attention ainsi que les projections de porte, montante et descendante du bloc neuronal. Le script prépare la base quantifiée pour l'apprentissage, désactive son cache et active le recalcul de certaines activations, ou *gradient checkpointing*, pour réduire la mémoire [98, 99].

Une fenêtre est la quantité de texte et de structures que le modèle peut traiter ensemble, comptée en jetons : mots, fragments de mots ou symboles. Instructions, définitions des outils, historique et observations occupent ce même espace. Le masque d'apprentissage indique quelles positions contribuent à l'erreur utilisée pour ajuster les paramètres : les observations fournissent le contexte et les réponses de l'assistant constituent les sorties à apprendre. Un masque incorrect pourrait ainsi entraîner le modèle à reproduire les résultats d'un outil au lieu d'apprendre à décider comment l'appeler.

Il s'agit d'un apprentissage supervisé sur des conversations : le contexte contient les consignes, les outils et leurs résultats ; la cible est la production attendue de l'assistant. Le gabarit définit un masque de génération qui inclut le texte et les appels d'outils de l'assistant, mais exclut les messages système, utilisateur et les observations. L'objectif n'est donc pas de mémoriser indistinctement tous les messages du protocole. La validation du masque constitue une condition du préflight, car un masque incorrect pourrait donner une perte apparemment normale en optimisant les mauvaises séquences [100, 102].

7.3 Quatre experts et un routage déterministe par phase

LANCE désigne cette organisation par HMoE, *Heterogeneous Mixture of Experts* dans le mémoire de conception. Le serveur porte aussi une dénomination historique *Hybrid*. Le mécanisme implémenté est, dans les deux cas, un routage agentique explicite : une requête complète reçoit un adaptateur choisi selon la phase. Aucun routeur neuronal appris ne distribue les jetons entre sous-réseaux et aucune combinaison pondérée des quatre experts n'est calculée [97, 103].

Les numéros de phase correspondent ici à l'analyse de topologie (1), la reconnaissance (2), l'analyse de vulnérabilités (3), la vérification (4), l'évaluation des accès (5) et la rédaction du rapport (6). Le secrétaire traite donc la préparation et la restitution, tandis que l'expert exploitation est employé pour vérifier des hypothèses et examiner des accès.

Table 6 – Répartition des experts et fenêtres du profil 3B configuré.

Expert	Phases	Fenêtre	Données et production visées	Risque à contrôler
Secrétaire	1, 6	6 144 jetons	Topologie et synthèse des artefacts de campagne	Ajouter des conclusions sans preuve.
Reconnaissance	2	6 144 jetons	Découverte, scans et services observés	Omettre un hôte ou inventer une version.
Vulnérabilités	3	4 096 jetons	Analyse par équipement et constats structurés	Confondre correspondance CVE et vulnérabilité vérifiée.
Exploitation	4, 5	6 144 jetons	Vérification contrôlée et état d'intrusion	Déclarer un accès sans observation directe.

L'identifiant commun `lance-moe` applique cette sélection. Les quatre identifiants `expert-*` permettent de forcer un expert pour le diagnostic ou une ablation. Le routeur recherche d'abord un numéro de phase dans le message système, puis des marqueurs de rôle précis. Si aucun marqueur ne correspond, il sélectionne la base. Un adaptateur demandé mais absent entraîne également un repli vers la base, journalisé par le serveur. Une requête réussie ne suffit donc pas à prouver que l'expert attendu était chargé : les journaux et l'état de santé doivent l'établir [103].

La frontière entre rôles facilite le remplacement indépendant d'un adaptateur et l'analyse des erreurs. Elle impose aussi une spécialisation rigide : une requête ambiguë n'associe pas spontanément plusieurs compétences. La sélection automatique doit ainsi être évaluée séparément de la qualité intrinsèque des adaptateurs.

7.4 Origine des corpus et préparation des conversations

Le mémoire de stage décrit quatre provenances. Le secrétaire utilise des scénarios synthétiques construits à partir des topologies, prompts et livrables LANCE. La reconnaissance transforme des observations Zeek du corpus IoT-23 en séquences compatibles avec les outils de découverte et de scan. Le corpus de vulnérabilités s'appuie sur un instantané NVD du 9 juillet 2026, croisé avec le catalogue CISA des vulnérabilités exploitées et la correspondance CVE d'Exploit-DB. Le corpus d'exploitation transforme des fragments de HackTricks, de comptes rendus communautaires Hack The Box et de gabarits Nuclei. Ces provenances sont *rapportées par le mémoire historique* : les téléchargements et fichiers bruts correspondants n'ont pas été réinspectés pour ce bilan. La présence archivée d'une ressource, par exemple BOTS v3 mentionné dans le mémoire, ne suffit pas à la compter parmi les entrées effectivement lues [97].

Le préparateur actuel prend des conversations JSONL, soit un objet JSON par ligne, et conserve la liaison entre un appel assistant et son résultat d'outil. Il réduit le catalogue aux outils utilisés dans la fenêtre, avec deux distracteurs par défaut. Les longues traces sont découpées aux frontières des tours ; les fenêtres suivantes gardent le préfixe et un marqueur de continuation. La phase 5 est traitée comme une chaîne de preuves atomique : sa conclusion doit rester avec les observations qui la soutiennent [101].

Lorsqu'une fenêtre dépasse encore le budget, le code réduit progressivement les contenus volumineux ; il peut ensuite compacter le milieu d'un message système. Les métadonnées conservent le modèle préparé, la fenêtre source et le signal de troncature ; un exemple toujours trop grand est écarté. Ce traitement constitue un compromis explicite : il limite la mémoire, mais peut enlever un indice sémantique utile. La conservation de la syntaxe des messages ne garantit pas la conservation de toute leur valeur probante [101].

7.5 Entraînement, validation et reproductibilité

Le profil 3B configure un lot unitaire, 32 étapes d'accumulation de gradient, un taux d'apprentissage de 10^{-4}, un échauffement de 3 %, une décroissance cosinus et l'optimiseur paginé AdamW en 32 bits. Reconnaissance et vulnérabilités ont deux époques prévues ; secrétaire et exploitation en ont une. La graine est 42. La partition réservée à la validation représente 2 % du corpus initial. Une époque parcourt le corpus d'entraînement ; l'accumulation combine plusieurs exemples avant une mise à jour des paramètres. Un checkpoint est un état sauvegardé du modèle. Le script prévoit une évaluation initiale et après chaque époque, puis restaure le checkpoint de plus faible perte de validation avant d'enregistrer l'adaptateur final. Ces valeurs décrivent la *configuration disponible*, sans attester qu'un entraînement précis l'a utilisée [98, 99].

Le préflight ne charge pas les poids. Il contrôle notamment les dépendances, les signatures attendues de la bibliothèque d'entraînement, l'environnement de calcul GPU CUDA et BF16, l'espace disque, les métadonnées des JSONL et le gabarit de conversation. Il peut refuser un processeur graphique déjà occupé. En mode de promotion, il vérifie les fichiers attendus des quatre adaptateurs et leur base déclarée. Une sortie existante est protégée : le lancement exige une reprise explicite ou un autre répertoire. Ces contrôles réduisent les erreurs de procédure ; ils ne mesurent pas l'efficacité d'un audit [102, 99].

Les workspaces séparent le code canonique et l'exécution GPU. Le premier porte les commits ; le second héberge données, environnement, checkpoints et adaptateurs. La synchronisation versionnée fonctionne en aperçu par défaut, avec liste blanche, copie atomique, empreintes SHA-256 et manifeste du commit, de la branche et de l'état local. Elle exclut structurellement les données, environnements et sorties du transfert de code. La collecte des rapports et celle des adaptateurs finaux sont distinctes et bornées. Ces chemins décrivent une procédure existante, sans constater la disponibilité actuelle de la machine d'exécution [105, 106].



Figure 6 – Cycle de spécialisation prévu par les outils du dépôt. La flèche de retour exige une acceptation humaine ; elle ne décrit pas un réentraînement automatique après chaque campagne.

7.6 Service d'inférence et limites de concurrence

Le serveur compatible OpenAI charge une base quantifiée et les adaptateurs PEFT, c'est-à-dire des modules d'ajustement à faible nombre de paramètres. Il expose des routes de santé, de catalogue et de conversation. Les experts ne sont pas quatre modèles indépendants dans un registre Ollama. Le changement d'adaptateur modifie un état partagé : le verrou englobe la sélection et la génération. Plusieurs requêtes peuvent être reçues, mais la génération sur cette instance est sérialisée. Cette protection assure la cohérence des poids ; elle ne fournit pas une capacité de génération parallèle et peut augmenter l'attente [103, 104].

L'état agentique est reconstruit à chaque requête à partir de la conversation : appels réalisés, erreurs de sauvegarde et exigences restantes. Le serveur n'entretient pas de mémoire persistante propre à une campagne. Il compacte l'historique dans les fenêtres de formation et réserve un budget de génération par expert. Pour la reconnaissance, les schémas visibles sont réduits à cinq outils centraux ; le registre exécutable complet demeure chez l'appelant. Lorsqu'une erreur fournit des

exigences structurées, une récupération déterministe peut demander l'outil manquant avant une nouvelle génération [103].

Ces décisions rendent l'intégration plus robuste, mais compliquent l'attribution scientifique : une amélioration peut venir de l'adaptateur, du compactage, de la réduction des outils ou d'une récupération déterministe. Les limites de contexte peuvent aussi supprimer une preuve utile. Enfin, le guide décrit une écoute restreinte et un accès privé parce que le serveur n'effectue pas de validation Bearer. Cette procédure documentée ne constitue pas un relevé de son exposition actuelle [104].

7.7 Boucle de correction supervisée et protection de l'évaluation

Le mineur d'erreurs compare les résultats aux vérités terrain de développement. Il produit des candidats dédupliqués et traçables, initialement en attente. L'attribution cherche la première phase disposant d'une cause observable : une cible absente de reconnaissance relève d'abord de `recon` ; un verdict d'exploitation contraire à une preuve négative relève de `exploit`. Une omission de topologie ou une synthèse désalignée peut corriger le secrétaire. L'export puis la conversion en exemples SFT, *Supervised Fine-Tuning*, exigent le statut `accepted` attribué après revue humaine [107, 108].

Les métadonnées, identifiants et catalogues de confiance doivent établir l'origine `dev-public`. Les groupes `test-public` et `eval-sealed` sont refusés, y compris lorsqu'un autre marqueur tente de les présenter comme développement. Une variante générée doit posséder une provenance vérifiable. L'import distant prévu utilise une liste fermée d'artefacts et un manifeste d'empreintes ; la compatibilité des commits permet d'éviter d'enseigner comme erreur du modèle une évolution de l'évaluateur [107, 109].

Le script ajoute le feedback après la séparation initiale entraînement/ validation. Le profil configure huit répétitions pour secrétaire, vulnérabilités et exploitation, zéro pour reconnaissance. Il faut donc vérifier le corpus réellement injecté dans le manifeste d'un run : la capacité de conversion multi-expert ne prouve pas que quatre corpus acceptés existent [98, 99].

La protection concerne également la mémoire documentaire. Le code interdit l'ingestion des résultats de benchmark dans l'historique et prévoit une source CVE figée pour l'évaluation officielle. Ces barrières ne prouvent pas l'absence historique de copies manuelles, de doublons entre fenêtres ou de variantes proches dans les partitions. Une indépendance par famille de traces et par scénario demeure nécessaire pour évaluer la généralisation [38, 37, 107].

7.8 État démontrable et expériences encore nécessaires

Le mémoire historique, révisé dans Git au commit `a98b480` du 28 août 2026, rapporte respectivement 19 065, 9 257, 6 951 et 14 629 exemples pour secrétaire, reconnaissance, vulnérabilités et exploitation. Il donne des pertes d'évaluation provisoires de 0,0421 ; 0,2102 ; 0,00475 ; 0,00836, dans ce même ordre. Ces valeurs constituent un *résultat historique rapporté*. La copie locale inspectée pour ce bilan ne contient ni `output/adapters/`, ni `output/training-workspace/`, ni rapport de préflight, ni `data/finetuning/`. Les poids, journaux du trainer et empreintes des corpus n'ont donc pas permis ici de reproduire ces chiffres ni d'identifier un modèle actuellement servi. Leur absence locale ne démontre pas l'absence d'entraînements sur un autre environnement [97].

Les acquis vérifiables sont une chaîne codée de préparation, de préflight et d'apprentissage, des configurations explicites, un service multi-adaptateurs, des frontières de synchronisation et une boucle de corrections humaines. Les tests dédiés vérifient ces outils ; ils n'exécutent pas une campagne GPU. Une faible perte signifie que les réponses du corpus réservé sont prévisibles pour le modèle. Les pertes de corpus différents ne sont pas directement comparables et ne prouvent ni un choix d'outil pertinent, ni une vulnérabilité correctement vérifiée [110, 97].

La validation à mener doit figer corpus, base, adaptateurs, routeur et code, puis comparer : base avec prompts seuls ; adaptateur global ; experts avec sélection correcte imposée ; HMoE automatique. Des ablations, comparaisons retirant un composant à la fois pour en isoler l'effet, peuvent retirer

le feedback, les récupérations déterministes ou permuter les experts. Les mêmes outils, budgets et scénarios doivent servir à chaque condition. Les mesures doivent couvrir validité des appels, achèvement des phases, précision, rappel, preuves, coût en jetons, latence et mémoire, avec plusieurs répétitions. C'est ce protocole, encore à établir avec des artefacts complets, qui permettra d'attribuer un gain à la spécialisation et de décider si son coût opérationnel est justifié [97].

8 Benchmark, méthode scientifique et résultats disponibles

8.1 Un corpus public composé, dont le test reste provisoire

IoTChainBench fournit l'instrument expérimental associé à LANCE. Son catalogue versionné 3.2.0 contient 29 scénarios publics : S1–S19 servent au développement et S20–S29 à l'évaluation d'une configuration figée. Les intitulés annoncent des situations à étudier, comme l'autorisation entre organisations, les mises à jour signées ou les parcours réseau à plusieurs relais ; ils ne certifient pas une capacité déjà démontrée par l'agent. S1h et S4h sont des variantes durcies historiques, hors des 29 identifiants officiels. Le support logiciel d'un groupe scellé existe, mais aucun scénario du catalogue n'y est inscrit [53, 48].

Quatre objets construisent une situation. La *topologie* décrit appareils, services, adresses et organisation réseau. Un *pack* regroupe des propriétés de sécurité, vulnérabilités et contrôles applicables à des rôles techniques. La *posture* sélectionne notamment des profils vulnérables, durcis ou mixtes. Le *scénario* compose ces éléments et ses objectifs ; sa *vérité terrain* énumère les failles et objectifs attendus pour l'évaluateur. Ainsi, S15 associe une identité légitime, une API vulnérable et des services protégés : une authentification fonctionnelle n'empêche pas nécessairement une lecture inter-organisation ou une élévation de droits applicatifs. Cette composition est plus informative qu'un simple nombre de machines [111].

Les 22 fichiers de définition de packs ne constituent pas autant de familles expérimentales indépendantes. Les scénarios peuvent partager composants, simulateurs, chemins ou méthodes. L'indépendance historique de S20–S29 vis-à-vis du choix des règles, instructions et budgets reste non vérifiée ; leurs données sont publiques. Une généralisation exigera donc des familles ou recompositions réservées avant le gel, un journal d'accès et des entrées publiques dépourvues d'indices de solution [3].

L'index d'étiquettes est dérivé des vérités terrain, avec leurs empreintes. Il n'apporte aucune observation supplémentaire : « logique » résulte d'une catégorie déclarée, tandis que surface, bruit et protocole décrivent uniquement les altérations effectivement consignées. L'audit statique du 12 octobre classe 215 des 288 vulnérabilités attendues dans des catégories à signature. Cette concentration motive le recentrage ; elle ne mesure pas les performances d'un automate. Enfin, cohérence YAML, syntaxe Ansible et déploiement correct restent trois vérifications différentes [112, 113].

8.2 Mesurer des populations, des preuves et des accès distincts

L'évaluation distingue les hypothèses candidates avant filtrage, celles retenues pour vérification et les confirmations déclarées. Ces dernières viennent des tests admis de phase 4, pas du texte libre du rapport. Chaque population est dédupliquée structurellement, puis appariée un-à-un à la vérité terrain ; une machine, un endpoint ou une condition différente ne doivent pas être fusionnés pour améliorer artificiellement le score. À l'étape finale, une correspondance exige également une preuve acceptée [68, 114].

Pour N prédictions distinctes, G failles de référence et V correspondances valides, les mesures sont

$$\mathrm{VP} = V, \quad \mathrm{FP} = N - V, \quad \mathrm{FN} = G - V, \qquad P = \frac{V}{N}, \quad R = \frac{V}{G}, \quad F_1 = \frac{2V}{N+G}.$$

La précision est indéfinie si $N = 0$, le rappel si $G = 0$. Sur un scénario positif, le score principal vaut $100F_1$; un résultat final évaluable mais vide vaut zéro, un résultat absent reste indisponible. Sur un contrôle sans faille, le F1 est remplacé par une spécificité au niveau du scénario : un si aucune fausse déclaration finale n'est présente, zéro sinon, sous condition d'analyse complète et de résultats évaluables. Un silence provoqué par une panne ne réussit donc pas le contrôle sain [68].

Une preuve doit soutenir la propriété et le périmètre annoncés : recevoir des noms de fichiers ne démontre pas leur contenu privé ; une bannière ne prouve pas une exploitation ; une connexion MQTT ne démontre pas la réception d'un message privé. Les contrats courants sont `strict-v3.16/evidence-v15`. Les traces acceptées, rejetées et manquantes restent séparées. Une déclaration soutenue mais hors référence conserve son statut non crédité et demande une revue : la vérité terrain peut être incomplète. Sans référence connue, un audit réel ne permet pas d'estimer exhaustivement les faux négatifs ou le rappel [68, 80].

La couverture de vérification rapporte le nombre d'hypothèses retenues ayant fait l'objet d'une tentative effective au nombre d'hypothèses de la file de vérification. Une planification ou un refus avant exécution ne constituent pas une tentative ; une observation produite puis non concluante reste comptée. Ce taux porte sur le travail engagé, tandis que précision et rappel portent sur la qualité des constats. Une couverture de 100 % peut coexister avec peu de confirmations ou des erreurs ; une file vide donne un taux indisponible. L'intrusion évalue séparément les accès et chemins. Plusieurs connexions directes ne prouvent pas un pivot : la provenance causale des transitions réseau manque actuellement, et les taux de pivot restent indisponibles. Aucun score global ne fusionne intrusion et F1 d'audit [68].

8.3 Attribuer un éventuel gain à une politique de décision

Comparer deux modèles dans le même pipeline répond au choix du modèle. Pour savoir si les décisions bénéficient d'un modèle, le protocole propose plusieurs politiques : D0, procédure fixe secondaire ; D1, automatisation conditionnelle compétente ; A0, plan produit par un LLM puis figé ; A1, agent révisant ses décisions après observation. Le sigle LLM désigne un grand modèle de langage. Une restitution D1-R, rédigée par modèle après clôture technique, pourrait isoler la qualité rédactionnelle sans modifier les résultats. D0, A0 et D1-R restent des comparateurs proposés, sans campagne réalisée attestée dans ce périmètre [3, 115].

L'incrément réellement branché compare D1, option `rules`, et A1, option `llm`, dans les phases 3 et 4 : analyse puis vérification. Il utilise le pipeline, les outils, l'exécuteur et les validateurs communs. Un inventaire public validé fournit les mêmes appareils et services aux deux bras. La comparaison ne mesure donc pas l'avantage d'une découverte autonome du réseau, de l'intrusion complète ou de la rédaction finale. Le mode impose le profil `full`, un worker, désactive la mémoire persistante et utilise une source CVE figée ; CVE désigne le registre d'identifiants de vulnérabilités [115, 116].

D1 réutilise le scanner déterministe et ses suivis conditionnels. En vérification, il applique les plans existants, une reprise admissible sur erreur transitoire et, si utile, une lecture HTTP structurée complémentaire : trois appels au plus par candidat dans cette boucle, hors scans. L'absence de plan reste une lacune explicite. A1 partage la base déterministe et ajoute analyses et actions du modèle. La référence n'est donc pas un script volontairement aveugle, mais sa couverture demeure bornée [115, 117].

Cette base contient aussi des connaissances des simulateurs : chemins, identifiants publics de test, rôles applicatifs et versions. Elles doivent être inventoriées pour éviter d'interpréter une familiarité avec les fixtures comme une généralisation. Les validateurs sont partagés : une relecture indépendante des observations brutes et de l'état cible demeure nécessaire pour éviter que la même erreur de validation ne crédite les deux bras [118].

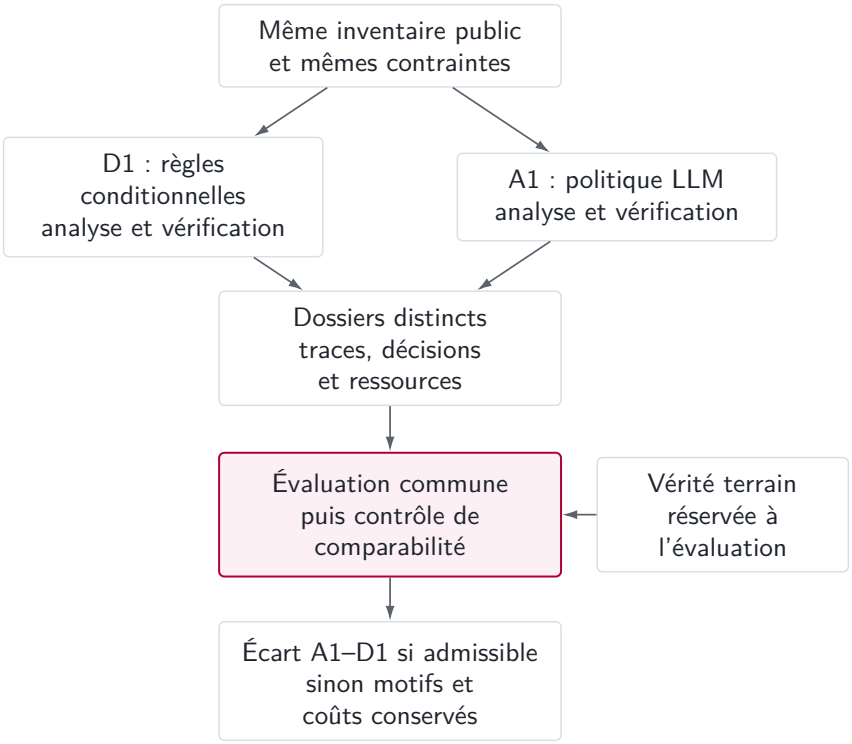


Figure 7 – Périmètre logiciel D1/A1 implémenté. L'égalité des déclarations d'état initial est vérifiée ; le reset réel reste à attester par l'opérateur. Le schéma ne représente pas une campagne achevée.

8.4 Appariement, budgets et prudence statistique

Une paire doit partager instance, inventaire, état initial déclaré, répétition, profil, workers, outils, ressources et contrats. Modèle, fournisseur et politique sont les différences attendues. L'empreinte des ressources couvre les contenus de `src/agent` et `src/benchmark`, y compris les changements locaux ; elle n'empreinte pas l'ensemble du système d'exploitation ou du laboratoire. Le comparateur exige les artefacts des trois populations, l'intégrité des preuves, une comptabilité complète et une analyse de tous les appareils annoncés. Il conserve les motifs d'incomparabilité et les résultats individuels [119].

Le manifeste décrit tous les essais prévus avant exécution, puis référence précontrôles et dossiers de résultats. Une empreinte d'état commune assure l'égalité d'une déclaration, sans certifier la remise à zéro réelle. Le bilan distingue toutes les issues opérationnelles et leurs consommations, puis les écarts techniques des seules paires admissibles. Cette séparation évite d'effacer les échecs du dénominateur, mais impose de publier les deux vues : une bonne qualité conditionnelle peut masquer une faible fiabilité [118, 120].

Le temps inclut les décisions du modèle. Les plafonds d'appels et financiers sont communs lorsqu'applicables ; zéro token ne signifie pas zéro coût matériel. Les limites sont coopératives et une opération déjà lancée peut dépasser l'échéance. Les coûts publiés ne couvrent ni infrastructure, ni maintenance, ni travail humain. Une consommation manquante n'est pas un zéro [115, 118].

Le code courant ajoute un bootstrap sur les moyennes par famille, avec intervalle à 95 % et correction de Holm entre métriques. Il requiert au moins deux familles renseignées ; ce minimum informatique n'établit ni indépendance ni puissance suffisante. L'analyse porte actuellement sur rappel, précision, F1 et spécificité, avec une marge d'amélioration par défaut de 0,05 sur ces mesures normalisées, soit cinq points. Cette marge est distincte du niveau statistique de décision. Le plan de soumission évoque pourtant un critère en vrais positifs confirmés et seuil préenregistré. Cet écart doit être résolu avant une conclusion confirmatoire. Les documents du 12 octobre annonçant l'inférence absente décrivent un état antérieur du code [121, 120, 122].

Une famille regroupe des situations apparentées qui ne doivent pas être comptées comme autant d'observations indépendantes. Le code moyenne d'abord les écarts A1–D1 au sein de chaque famille, puis rééchantillonne ces moyennes avec remise pour estimer la variabilité de l'écart global. L'intervalle à 95 % exprime cette incertitude sous les hypothèses de la méthode ; il ne garantit pas le même écart sur toute infrastructure IoT. La correction de Holm limite le risque de retenir un résultat favorable parce que plusieurs métriques ont été examinées. Les cinq points désignent, eux, l'ampleur minimale de gain jugée utile. Avec peu de familles, l'incertitude et leurs dépendances limitent fortement l'interprétation.

8.5 Campagnes historiques : résultats et défauts observés

Le premier lot réel Qwen UMONS commence le 4 octobre 2026 avec `qwen3.8:27b`, six phases et profil `full`. Son bilan gelé le 5 octobre à 11 :32 s'appuie sur quatorze archives et 1 582 empreintes contrôlées dans le rapport d'origine. Les scores historiques utilisent `strict-v3.14/evidence-v13`. Le S1 commun obtient $F_1 = 0,880$, avec 11 vrais positifs, deux faux positifs et un faux négatif. Deux diagnostics S1 distincts, à 0,917 et 0,846, sont exclus du lot. S12 échoue sans score final ; son absence ne devient pas zéro [123].

Pour lire ces chiffres, les 11 vrais positifs, deux faux positifs et un faux négatif du S1 correspondent à 13 constats finaux pour 12 failles de référence. La précision vaut 11/13, le rappel 11/12 et le F_1 vaut 22/25, soit 0,880. Ce score ne signifie pas que 88 % du réseau a été compromis : il résume les constats appariés sous le contrat historique. Les accès obtenus, les erreurs d'exécution et la surface non observée restent des dimensions différentes.

Les autres F1 transcrits sont S2 : 0,750 ; S3 : 0,545 ; S4 : 0,450 ; S5 : 0,541 ; S6 : 0,686 ; S7 : 0,552 ; S8 : 0,483 ; S9 : 0,538 ; S10 : 0,333 ; S11 : 0,481. Cette dispersion décrit des observations de développement, sans répétitions appariées contre D1. Des réserves concernent les compteurs de S1 et le crédit de chemins de S6 ; les faux négatifs, preuves secondaires et propriétés de référence n'ont pas été certifiés exhaustivement [123, 124].

Quatorze corrections générales ont ensuite motivé une campagne distincte, sur la référence `0f4e5ff`. Les validations hors réseau et la CI ne réécrivent pas les scores initiaux. Le diagnostic du 6 octobre à 09 :29 classe S2/S3 terminés, huit runs partiels et S7/S12 échoués. S7 manque un champ structurel lors de l'agrégation ; S12 subit une génération de graphe tronquée en phase 1, différente de l'échec initial. Erreurs scanner et délais modèle expliquent les statuts partiels observés sans établir une cause commune [125, 126].

Table 7 – Chronologie des observations conservées. Les contrats et populations diffèrent ; aucune tendance de performance entre lignes n'est calculée.

Périmètre et date	Observation consignée	Portée et statut
Lot initial, gel 05/10 11 :32	11 scores ; S12 indisponible ; F1 de 0,333 à 0,880	Développement, `strict-v3.14` ; réserves sémantiques, sans comparaison D1.
Correctif, 06/10 09 :29	2 terminés, 8 partiels, 2 échoués	Objectif S1–S12 non atteint ; lecture de métadonnées sélectionnées.
Essai préparatoire S1, 14/10	66,7 % règles ; 72,0 % LLM	Journal : paire comparable, VP confirmés +1 ; essai d'instrument exclu du pilote gelé.
Autre essai S1, 14/10	66,7 % dans les deux bras	Ancienne empreinte de ressources ; conservé hors des 12 paires.
Manifeste local, 15/10	12 paires prévues, zéro dossier renseigné	Aucune paire admissible enregistrée ; tous précontrôles en attente.

8.6 Pilote D1/A1 : budgets calibrés, campagne non renseignée

Le pilote prépare S1, S14, S15 et S1h, trois répétitions par situation, soit douze paires et vingt-quatre bras. L'ordre est équilibré globalement : six paires D1 puis A1 et six dans l'ordre inverse, sans randomisation. Il n'est pas équilibré au sein de chaque situation. Son modèle local UMONS est déclaré `qwen3.8:27b`, avec digest Ollama enregistré : taille 27,3 milliards de paramètres et quantification Q4_K_M selon le précontrôle. Le manifeste courant fixe 200 appels, 1 800 secondes, 600 secondes par appareil en analyse et un plafond monétaire comptabilisé de 2 dollars [127, 128].

Le journal des 13–14 octobre conserve les essais ratés : droits de lancement, vault interactif, noms de services non reconnus, double verrou bloquant les agents et artefacts manquants. À 600 secondes, le bras LLM atteint sa limite avant de terminer les vérifications S1. La calibration à 1 800 secondes s'applique ensuite aux deux bras, avant l'admission des douze paires. Ces corrections de mécanique ne sont pas présentées comme un avantage scientifique [128].

Un essai de préparation, aussi appelé essai de fumée, vérifie le branchement de l'instrument avant la campagne ; ses résultats servent à la calibration. Pour l'essai `38549458127`, le journal rapporte une paire comparable : F1 confirmé 66,7 % contre 72,0 %, un vrai positif confirmé supplémentaire et six faux positifs *candidats* supplémentaires côté LLM. Ce dernier consomme 1 394 secondes, 1,34 dollar comptabilisé et 1 189 300 tokens. Une autre paire S1 est annoncée à égalité, 66,7 %. Ces nombres sont attribués au journal ; ils ne résultent pas d'un nouveau calcul ni d'une nouvelle inspection exhaustive de leurs traces [128].

Un changement de l'instrument de bilan modifie ensuite l'empreinte des ressources. Les essais antérieurs sont conservés hors du pilote gelé. Dans la copie locale consultée, les vingt-quatre chemins de bras restent nuls et les précontrôles `pending`. Aucun résultat de campagne admissible n'y est donc enregistré. Un workflow réussi ou un prochain identifiant d'essai ne comble pas ce manque ; le bilan doit suivre les preuves effectivement référencées [127, 128].

8.7 Plan V2 : rendre les questions et les mesures explicites

La proposition V2 révisée le 15 octobre décrit 44 fiches `s_...`, 25 ensembles techniques, 216 configurations sélectionnées et 456 bras conditionnels par instance et répétition communes ; 24 configurations incluent A0. Une fiche décrit une situation, un type son traitement, un bras la politique, une instance un monde paramétré et une répétition un nouvel essai après reset. Ces nombres ne sont ni des exécutions accomplies, ni des familles indépendantes à additionner au corpus public [129].

Ces unités se comprennent sur `s_ssh_identifiants` : la fiche pose la question des identifiants faibles ; deux configurations retiennent une posture protégée et une posture vulnérable. Pour une même instance, c'est-à-dire les mêmes paramètres de cible et d'objet témoin, chaque configuration est exécutée avec D1 et A1 dans l'ordre déclaré. Cela donne quatre bras par instance et répétition communes. Une nouvelle répétition exige une remise à zéro et ne crée pas une nouvelle famille. Les 456 bras comptent des traitements prévus avant leur multiplication éventuelle par le nombre d'instances et de répétitions ; ils ne représentent pas 456 réseaux ni 456 résultats acquis.

Six questions organisent la lecture : établir une preuve ; comprendre contexte et droits ; interpréter un contrat nouveau ; suivre état et séquence ; planifier un parcours contraint ; réviser après changement. Elles ne définissent pas six scores cognitifs. Les cinq tâches du tableau 8 fixent des unités différentes. Un cas protégé exige des contrôles réellement exécutés et un témoin légitime fonctionnel ; l'abstention peut être correcte lorsque les observations sont insuffisantes. Les comparaisons hors ligne emploient un interpréteur à règles et un interpréteur LLM sur le même dossier figé [129, 130].

L'ambition réseau reste à qualifier : 35 fiches sont uniquement locales et 36 uniquement à carte complète.

Local signifie que la cible est directement accessible depuis le point d'entrée autorisé, sans relais réseau à organiser. Une carte complète signifie que l'architecture utile est fournie dès le départ.

Table 8 – Cinq tâches V2 proposées ; les mesures sont à calibrer puis geler avant l'évaluation.

Tâche	Mesure primaire proposée	Exemple et précaution
M-T : ciblée	Proportion d'essais avec objectif évalué et preuve valide au budget	SSH faible ou lecture MQTT privée ; contrôler aussi la variante protégée.
M-G : audit global	Nombre de défauts confirmés par essai au budget fixé, zéros inclus	Sauvegardes privées ; déclarer erreurs, couverture et coûts.
M-I : interprétation	Proportion de conclusions conformes au dossier, abstention incluse	Preuves contradictoires ; aucune exploration réseau n'est mesurée.
M-E : état	Proportion de diagnostics corrects de l'état observable	Interruption OTA ; ne pas demander de deviner un état indiscernable.
M-C : couverture	Proportion de diagnostics étayés dans le graphe autorisé fini	Cible inaccessible ; distinguer accès, évaluation et impossibilité de conclure.

Ces situations peuvent conserver des difficultés de preuve, de droits ou de séquence, sans mesurer la découverte d'une architecture inconnue. Les deux nombres décrivent des axes différents : une fiche peut être à la fois locale et à carte complète. Ils ne doivent donc pas être additionnés. Les limites d'accès, resets, coûts, chemins témoins et oracles restent à attester. Le plan ne constitue donc pas une nouvelle campagne déployée. Le programme de soumission doit être aligné sur ces tâches : un critère générique en vrais positifs ne couvre pas automatiquement un diagnostic d'état ou d'interprétation. Une égalité ou une dégradation du LLM demeure une issue recevable [129, 122].

8.8 Premier incrément SSH et conditions de passage à la mesure

L'exemple SSH cherche à établir une lecture privée obtenue avec un couple utilisateur–mot de passe faible, au-delà d'un port acceptant une connexion. La fixture est l'environnement de test préparé ; le canari est un contenu témoin propre à l'essai ; l'oracle est le dossier privé indiquant la posture attendue et l'empreinte de ce contenu. Un compte légitime vérifie que le serveur et l'objet sont fonctionnels. La politique reçoit la cible, ce compte, le dictionnaire public et le chemin autorisé, mais ni le couple gagnant éventuel ni le contenu attendu. L'évaluateur peut ainsi contrôler l'observation sans fournir sa réponse à la politique [131, 132].

Le premier incrément V2 implémente un contrat public SSH, un adaptateur D1 réutilisant l'exécuteur et un évaluateur hors ligne. Il réalise un contrôle légitime puis parcourt un dictionnaire public fini ; identité effective et lecture du même objet doivent être observées ensemble. L'oracle privé contient la posture attendue et l'empreinte d'un canari propre à l'essai. Pour le cas protégé F0, tous les couples doivent être explicitement refusés avec un témoin fonctionnel. Pour le cas vulnérable F1, l'accès faible et la lecture privée doivent être prouvés. Ici F1 désigne la posture, distincte du score F_1 [131, 132].

La sortie annonce toujours `mt_admissible:false` : c'est un verdict probatoire, pas encore une mesure ciblée admise. La fixture réelle F0/F1, le raccordement A1, la parité des budgets, le reset et le gel expérimental restent à produire. La validation consignée le 15 octobre rapporte 32 tests SSH réussis et 3 885 tests réussis, un ignoré, dans la suite complète. Les processus SSH y sont simulés. Ces résultats antérieurs sont cités ; aucun test n'a été relancé pour ce bilan [131].

La CI prévoit validation des compositions, syntaxe Ansible, tests et contrôle de l'image isolée. Elle qualifie des contrats logiciels, sans prouver toutes les propriétés des services déployés. La priorité est de fermer ce premier contraste, puis MQTT lecture, autorisation API, reprise MQTT et cache inter-identités. Avant toute revendication, il faut enfin réconcilier manifeste, ressources,

mesures et analyse statistique, archiver les preuves et revoir les désaccords. Le passage aux scénarios suivants dépend d'une qualification technique, sans exiger une victoire du LLM [60, 129].

9 Avancement transversal et portefeuille de recherche

9.1 Une progression à lire par livrables

Le projet a produit une chaîne technique qui permet de décrire un réseau, d'orchestrer des contrôles, de conserver des observations, d'appliquer un évaluateur et de préparer l'apprentissage à partir des erreurs. Ces réalisations sont reliées : la qualité des données d'entraînement dépend des résultats d'audit ; la valeur d'un score dépend de l'environnement et des preuves ; l'utilité de l'interface dépend de l'accès à ces éléments.

Le tableau 9 rassemble les dimensions d'avancement. Il distingue les réalisations présentes dans le dépôt des validations encore nécessaires : la présence du code ne démontre ni robustesse générale ni avantage expérimental.

Dimension	Réalisation vérifiable dans le dépôt	Validation encore nécessaire
Modélisation et moteur	Graphe, contexte par run, six phases, outils, profils, artefacts et suivi de consommation.	Robustesse sur des configurations et tailles variées, avec preuves de couverture conservées.
Laboratoire	Catalogue, topologies, packs, playbooks, précontrôles et réservation partagée.	Qualification des propriétés réellement déployées, remises à zéro et scénarios retenus.
Sécurité	Contrats d'attribution, validations de preuve, statuts et frontières d'exécution.	Revues de preuves et analyse des cas non couverts sans généraliser les garanties.
Application	Lancement, suivi, arrêt, historique, configuration et parcours de comparaison.	Évaluation des usages et du travail humain sur tâches définies.
Modèles	Préparation, entraînement paramétrable, service multi-adaptateurs et routage par rôle.	Artefacts d'entraînement à vérifier ; comparaison du modèle de base et des spécialisations sur données indépendantes.
Évaluation	Évaluateur commun, métriques, comparateur apparié et bilan de campagne.	Campagne complète gelée et analyse alignée avec les hypothèses préenregistrées.

Table 9 – Maturité qualitative des ensembles du projet. Les conditions de validation indiquent ce qu'il reste à établir, sans score global d'achèvement.

Le rapport de jalon III du programme G6259 présente la méthodologie de test M2.3 et la plateforme virtuelle parmi les contributions de suivi. La question scientifique d'un avantage du LLM constitue une preuve supplémentaire à construire. L'achèvement déclaré d'un livrable méthodologique peut donc coexister avec des campagnes expérimentales et des validations spécialisées encore en cours. Le présent bilan ne réévalue pas les jalons contractuels des partenaires ; il précise ce que les éléments locaux permettent d'examiner [1].

9.2 Les recherches qui accompagnent le développement

Plusieurs études soutiennent les évolutions du logiciel. Elles ont leur question et leur statut propres ; leurs propositions ne sont pas toutes des fonctionnalités livrées. L'étude sur le contexte cherche à conserver les observations, les contradictions et les décisions utiles dans les limites de requête. Les reproductions synthétiques du 29 septembre ont mis en évidence des pertes possibles ; leur fréquence et leur effet sur les performances réelles restent à mesurer [133].

L'étude sur les méthodes d'audit examine une organisation plus explicite des propriétés, de la portée et des obligations de preuve. Elle propose des comparaisons hors ligne entre représentations

des mêmes observations et une relecture sémantique bornée. Ces propositions peuvent améliorer la façon de discuter un résultat sans démontrer par elles-mêmes une meilleure détection réseau [134].

Le diagnostic d'organisation du code du 3 octobre a formulé des lots de réorganisation et des critères d'acceptation. Son index indique ensuite une mise en œuvre partielle, notamment des contextes de phase isolés. Les mesures et descriptions de l'ancienne copie restent historiques ; l'architecture actuelle se lit dans le code et les contrats courants. Cette distinction permet de conserver le raisonnement du diagnostic sans maintenir deux descriptions concurrentes du système [135].

Le recentrage du corpus développe une carte de situations où l'interprétation, la planification et la révision peuvent avoir des effets différents. Il affine les unités de mesure au lieu de traiter tous les scénarios comme une difficulté unique. Ce travail de conception et le premier incrément SSH constituent des livrables ; la réalisation complète des environnements et des comparaisons reste à conduire : serveurs réels, témoins vulnérables et protégés, remise en état, raccordement de la politique LLM et campagne appariée [129].

Une spécialisation vers l'Internet des objets médicaux (IoMT) a également été formulée comme sujet de travail. Le dossier est explicitement au statut de proposition non démarrée. Il envisage des réseaux médicaux simulés et une évaluation par rapport à la version généraliste. Il ne démontre ni support clinique actuel ni sûreté sur dispositif médical réel. Cette piste élargit les possibilités de recherche ; elle ne doit pas être comptée parmi les capacités déjà qualifiées [136].

10 Dépendances, limites et priorités de poursuite

10.1 Le chemin critique relie environnement, preuve et décision

Le principal enjeu transversal est l'attribution. Une dégradation peut provenir du laboratoire, de l'inventaire, du scanner, d'une décision, d'un budget ou de l'assemblage du résultat. Pour corriger le bon composant et comparer deux politiques, il faut conserver l'endroit où l'exécution s'est arrêtée et les observations réellement disponibles.

La figure 8 représente la dépendance minimale d'une conclusion comparative. Chaque étape possède ses livrables et son contrôle. Une augmentation du nombre de scénarios ou de modèles ne compense pas un défaut d'état initial, une preuve mal attribuée ou un résultat manquant.

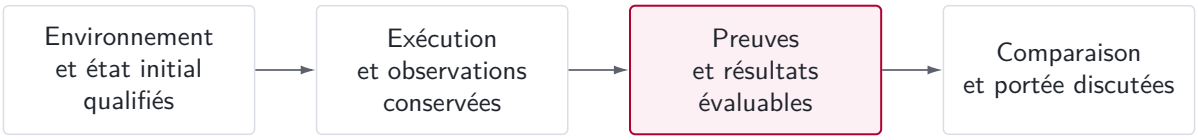


Figure 8 – La validité d'une comparaison repose sur une chaîne de dépendances. Les performances d'une politique sont interprétables lorsque les conditions qui rendent sa mesure valide sont établies.

Les contraintes de ressources sont également transversales. Une requête longue peut réduire le temps restant pour la vérification ; une projection trop agressive peut masquer une observation ; une reprise peut augmenter la consommation tout en permettant un livrable valide. Le compromis entre coût, latence et couverture doit être mesuré sur des configurations identifiées, avec les tentatives infructueuses conservées.

10.2 Un programme de travail par critères de sortie

Les priorités du tableau 10 sont proposées à partir du bilan. Elles concernent autant la plateforme que les expériences. Un critère de sortie permet d'annoncer une avancée concrète et d'éviter une progression estimée à partir du seul volume de développement.

Priorité	Travail à mener	Critère de sortie proposé
État reproductible	Réconcilier version, ressources, inventaire, état initial et manifeste.	Une exécution peut être reliée à une configuration et à des précontrôles archivés.
Qualification	Vérifier les propriétés des instances retenues et leur remise à zéro.	Témoins vulnérables/protégés et effets attendus réellement contrôlés.
Preuves et revue	Compléter les dossiers et examiner les cas rejetés ou indéterminés.	Conclusions attribuées aux bonnes propriétés, avec réserves et contre-preuves visibles.
Campagne	Exécuter les essais déclarés et publier leur bilan complet.	Bras présents, absents et échoués comptés ; analyse alignée au protocole gelé.
Spécialisation	Comparer modèle de base, adaptateurs et routage sur tâches communes.	Qualité technique, fiabilité, latence et coûts attribuables à la configuration étudiée.
Usage et partage	Qualifier les parcours et préparer un extrait reproductible.	Procédure exécutable, limites de portabilité explicites et retour d'utilisateurs sur tâches définies.

Table 10 – Feuille de route proposée pour consolider conjointement logiciel, expérimentation et usages. Chaque critère demande des configurations et des observations archivées.

La calibration du pilote, l'exécution de ses douze paires et la qualification du premier scénario SSH V2 sont deux travaux liés mais distincts. Une campagne doit conserver sa configuration pendant l'exécution ; les nouveaux scénarios, outils et modèles doivent être introduits dans des lots identifiés, avec leur propre gel. La préparation des recherches futures peut continuer à condition de préserver cette attribution [127, 131].

10.3 Publication et diffusion

Le dossier d'objectifs retient une décision interne le 16 novembre 2026, puis une soumission visée à AsiaCCS 2027 le 11 décembre 2026. Ces dates sont les jalons consignés du projet ; elles ne sont pas une garantie de résultats ni une vérification nouvelle du calendrier de la conférence. Les contributions envisagées portent sur le benchmark, les conditions d'écart et le coût de l'avantage éventuel. Un résultat nul mais mesuré avec une précision suffisante peut conduire à repositionner la contribution [122].

La diffusion exige aussi un artefact compréhensible hors de la machine de développement. Le laboratoire Proxmox, les fournisseurs locaux et certaines ressources matérielles limitent la portabilité. Un extrait reproductible doit préciser ce qu'il reproduit, ce qu'il simule et ce qui nécessite l'infrastructure de référence. Les sources, traces, configurations et modes d'emploi sont donc des livrables de recherche au même titre que les tableaux de résultats.

11 Conclusion générale

LANCE rassemble plusieurs réalisations qui répondent à une même difficulté : transformer des décisions de modèles et des contrôles réseau en résultats examinables. Le moteur en six phases organise le travail ; le graphe décrit les relations et soutient la priorisation ; le laboratoire produit des situations contrôlées ; les contrats de preuve délimitent les conclusions ; l'application rend l'exécution consultable ; l'entraînement prépare des modèles adaptés aux rôles ; le benchmark fournit le cadre de comparaison.

L'avancement dépasse donc la seule rédaction d'un protocole. Des composants logiciels, des scénarios, des essais historiques, des diagnostics et des outils de qualification existent. Leur portée reste différente : des tests simulés ne remplacent pas une qualification de laboratoire, une représentation de chemin potentiel ne prouve pas une intrusion et une observation isolée ne permet pas une conclusion générale sur le LLM.

La suite consiste à relier ces acquis dans des campagnes complètes, à qualifier la spécialisation et à rendre les résultats reproductibles. L'objectif est de pouvoir expliquer quelles propriétés ont été examinées, quelles preuves ont été obtenues, où l'exécution a rencontré ses limites et dans quelles conditions les modèles apportent un bénéfice utile. Le projet dispose des principaux instruments pour poser ces questions ; leur réponse demande encore des mesures contrôlées et une discussion de leur portée.

Références

[1] Programme NATO SPS G6259. G6259_Milestone3_UMONS_FINAL.docx. Rapport local de jalon III du 1er septembre 2026 ; contexte institutionnel déclaré.

[2] Projet LANCE. README.md. Objectifs et périmètre actuels, consultation du 15 octobre 2026.

[3] Projet LANCE. research/2026-09-27-agent-vs-automation/protocol.md. Protocole proposé et comparaison des politiques ; copie locale au 15 octobre 2026.

[4] Projet LANCE. metric/squelette-latex/parts/partie1.tex. Contexte et mission de spécialisation HMoE ; document local consulté pour l'historique des objectifs.

[5] Équipe du projet. report/q4-2025.tex. Rapport historique Q4 2025 ; architecture de l'époque, sans reprise des performances comme résultats actuels.

[6] Équipe du projet. report/q1-2026.tex. Rapport historique Q1 2026 ; trajectoire architecturale déclarée, distincte de la version courante.

[7] Projet LANCE. src/agent/phases/README.md. Organisation courante des phases et contextes ; lecture du 15 octobre 2026.

[8] Projet LANCE. research/README.md. Index et statut des études ; consultation du 15 octobre 2026.

[9] Projet LANCE. src/agent/pipeline.py. Cycle de vie, prérequis, choix de modèle, finalisation et mémoire ; lecture statique du 15 octobre 2026, sans exécution.

[10] Projet LANCE. src/agent/core/runner.py. Boucle de phase, résolution des outils et transactions de sauvegarde ; code local inspecté le 15 octobre 2026.

[11] Projet LANCE. src/graph_backend.py. Interface GraphBackend et implémentation NetworkXBackend ; sémantique des requêtes examinée le 15 octobre 2026.

[12] Projet LANCE. src/db/database.py. Schéma SQLite des fournisseurs, modèles, runs, scores et consommations ; lecture statique du 15 octobre 2026.

[13] Projet LANCE. src/agent/phases/context.py. PhaseContext, copie des variables et services explicites ; code local au 15 octobre 2026.

[14] Projet LANCE. src/agent/registry.py. Responsabilités, prérequis, groupes d'outils et noms de livrables des phases ; confrontés aux modules courants le 15 octobre 2026.

[15] Projet LANCE. src/loader.py. Chargement YAML des objets d'infrastructure et choix de NetworkX par défaut ; lecture du 15 octobre 2026.

[16] Projet LANCE. src/risk_scorer.py. Politique heuristique directed-v2, exposition, centralité et combinaison des composantes ; code local du 15 octobre 2026.

[17] Projet LANCE. src/attack_path.py. Pondération heuristique et chemins dirigés calculés par Dijkstra ; mécanisme présent, distinct d'une observation de pivot.

[18] Projet LANCE. src/cve_lookup.py. Recherche NVD et classification produit/version/CPE ; code inspecté le 15 octobre 2026, aucune interrogation distante pour ce rapport.

[19] Projet LANCE. src/agent/tools/disbalance_engine.py. Graphe pondéré, coûts et comparaison des distances ; modèle de priorisation, non résultat expérimental.

[20] Projet LANCE. src/agent/tools/graph_tools.py. Parcours infrastructure, scénario public et découverte ; outils de consultation et absence de précalcul risque/chemins en scénario public.

[21] Projet LANCE. src/agent/core/graph_context.py. État de graphe par run, liaison des workers et session autonome de compatibilité ; lecture du 15 octobre 2026.

[22] Projet LANCE. src/agent/phases/recon/run.py. Projection des observations et contrôles de couverture de reconnaissance ; code local au 15 octobre 2026.

[23] Projet LANCE. src/agent/phases/analysis/run.py. Coordination scanner, analyses par équipement et agrégation ; lecture statique du 15 octobre 2026.

[24] Projet LANCE. src/agent/phases/analysis/aggregation.py. Capture du contexte et agrégation conservant le registre brut ; code local au 15 octobre 2026.

[25] Projet LANCE. src/agent/phases/verification/run.py. Workers de vérification et reconstruction des statuts depuis les traces ; lecture du 15 octobre 2026.

[26] Projet LANCE. src/agent/phases/intrusion/run.py. Contexte et clôture de la phase 5, garde contre l'absence de tentative ; code local inspecté le 15 octobre 2026.

[27] Projet LANCE. docs/architecture/report-writing.md. Rédaction par fiches et assemblage factuel ; limites conservées, anciens délais non utilisés comme contrat courant.

[28] Projet LANCE. src/agent/phases/report/run.py. Rédaction séquentielle, réutilisation des fiches et délai adapté à leur nombre ; code du 15 octobre 2026.

[29] Projet LANCE. src/agent/prompt_manager.py. Chargement des prompts, inclusions et interpolation ; lecture statique du 15 octobre 2026.

[30] Projet LANCE. src/agent/tools/tool_loader.py. Schémas YAML, adaptateurs, suggestions matérielles et filtrage de disponibilité ; code courant, sans vérification d'équipement réel.

[31] Projet LANCE. src/agent/provider.py. Adaptateur de fournisseurs compatibles OpenAI et boucle synchrone d'appels d'outils ; lecture du 15 octobre 2026.

[32] Projet LANCE. src/agent/core/executor.py. Exécuteur instrumenté, budgets et politiques de connaissances ; code local au 15 octobre 2026.

[33] Projet LANCE. src/agent/execution_profiles.py. Profils compact/full, routage des outils et résolution auto au seuil de 32 milliards ; configuration inspectée le 15 octobre 2026.

[34] Projet LANCE. src/agent/phases/recon/subagents.py. Balayage, lots de couverture et fusion depuis le journal ; conditions d'éligibilité examinées le 15 octobre 2026.

[35] Projet LANCE. docs/architecture/phase1-graph-recovery.md. Reprise de rédaction après troncature en phase 1 ; mécanisme présent, sans nouveau rejeu expérimental.

[36] Projet LANCE. docs/architecture/phase3-block-recovery.md. Reprise par blocs de services et conditions de promotion ; documentation consultée le 15 octobre 2026.

[37] Projet LANCE. src/agent/tools/skill_tools.py. Fiches, recherche sémantique, classification CVE et instantané du benchmark ; lecture du 15 octobre 2026.

[38] Projet LANCE. src/agent/knowledge/ingest.py. Ingestion de méthodes et de constats ; exclusion des artefacts benchmark de la mémoire épisodique, lecture du 15 octobre 2026.

[39] Projet LANCE. src/agent/core/provider_transport.py. Reprises de transport et délais distincts des continuations modèle ; code inspecté le 15 octobre 2026.

[40] Projet LANCE. src/agent/core/provider_concurrency.py. Admission des requêtes par origine et libération du verrou ; code local du 15 octobre 2026.

[41] Projet LANCE. src/agent/cost_tracker.py. Mesures par phase, provenance des tarifs, durée cumulée et écoulée, budget de consommation observée ; lecture du 15 octobre 2026.

[42] Projet LANCE. src/agent/knowledge/store.py. Client ChromaDB persistant, recherche et repli éphémère ; code inspecté le 15 octobre 2026, sans ouverture d'une base pour ce rapport.

[43] Projet LANCE. src/agent/phases/registry.py. Routage explicite des six modules de phase ; code local inspecté le 15 octobre 2026.

[44] Projet LANCE. AGENTS.md. Cible nato/pve-nato, runner nato-master et exclusion du homelab personnel ; instructions locales lues le 15 octobre 2026.

[45] Projet LANCE. benchmarks/ansible/playbooks/deploy_master.yml. Provisionnement du contrôleur LXC, interfaces, filtrage, Tailscale et service applicatif ; lecture statique ciblée le 15 octobre 2026. Aucun état distant vérifié.

[46] Projet LANCE. benchmarks/ansible/playbooks/03_deploy_scenario.yml. Clonage, appartenance des charges, conflits, interfaces VLAN et préparation réseau ; lecture statique le 15 octobre 2026.

[47] Projet LANCE. docs/guides/lab-access-tailscale.md. Accès au laboratoire ; procédure documentée comme vérifiée le 7 octobre 2026, relue le 15 octobre sans nouvelle connexion.

[48] Projet LANCE. docs/benchmark/v1/scenarios.md. Corpus, séparation des groupes, composition, exposition de la topologie et limites de déploiement ; état local consulté le 15 octobre 2026.

[49] Projet LANCE. docs/benchmark/catalogue-attaques.md. Guide de couverture et limites des simulations ; relecture documentaire datée du 15 septembre 2026, confrontée aux fichiers locaux le 15 octobre.

[50] Projet LANCE. benchmarks/topologies/true_multihop.yaml. Objectif S20 à deux relais et segments isolés ; définition locale lue le 15 octobre 2026, sans essai réseau.

[51] Projet LANCE. benchmarks/ansible/playbooks/06_verify.yml. Contrôles de propriétés, refus attendus, chemins réseau et consolidation du preflight ; lecture statique ciblée le 15 octobre 2026, sans exécution.

[52] Projet LANCE. benchmarks/packs/definitions/f15_ot_protocols.yaml. Propriétés et contrôles des simulateurs OT et registre Modbus jetable ; définition locale consultée le 15 octobre 2026.

[53] Projet LANCE. benchmarks/catalog.yaml. Catalogue 3.2.0 : 29 scénarios publics et groupes ; lecture complète le 15 octobre 2026.

[54] Projet LANCE. docs/guides/manual-scenarios.md. Composition manuelle, profils, adaptateurs d'exécution, prévisualisation et sémantique des chemins ; lecture ciblée le 15 octobre 2026.

[55] Projet LANCE. src/benchmark/scenario_deployment.py. Adaptation des exports aux playbooks, surcouches et réservations d'identifiants ; lecture ciblée le 15 octobre 2026.

[56] Projet LANCE. src/agent/core/lifecycle.py. Cycle déploiement–injection–vérification, contrat preflight-v1 et nettoyage ; lecture ciblée le 15 octobre 2026.

[57] Projet LANCE. benchmarks/ansible/playbooks/08_reset_scenario.yml. Redémarrage, réinjection et repeuplement ; lecture complète le 15 octobre 2026, sans reset exécuté.

[58] Projet LANCE. docs/guides/development-instances.md. Trois emplacements applicatifs, réservation commune, activité déclarée, CI et limites ; état documentaire local relu le 15 octobre 2026.

[59] Projet LANCE. src/benchmark/lab_lock.py. Réservation coopérative interprocessus du laboratoire ; lecture ciblée le 15 octobre 2026.

[60] Projet LANCE. .github/workflows/benchmark-integrity.yml. Validation complète ou documentaire et conditions d'appel du déploiement ; état local lu le 15 octobre 2026.

[61] Projet LANCE. .github/workflows/update-master.yml. Déploiement du commit validé sur le runner nato-master ; état local lu le 15 octobre 2026. Le script scripts/deployment/deploy-instance.sh a également été consulté pour ses contrôles d'identité et d'emplacement.

[62] Projet LANCE. src/agent/finding_identity.py. Identité conservatrice des constats et séparation des identifiants de provenance ; lecture statique du 15 octobre 2026.

[63] Projet LANCE. src/benchmark/evaluator.py. Chargement des traces et attribution des preuves, notamment `_tool_call_matches_finding` et `_assign_evidence_refs` ; lecture ciblée du 15 octobre 2026, sans calcul nouveau.

[64] Projet LANCE. src/agent/phases/verification/contract.py. Plans de vérification par famille et routage des outils ; lecture ciblée du 15 octobre 2026.

[65] Projet LANCE. src/agent/exploit_evidence.py. Interprétation partagée des traces et synthèse des verdicts ; lecture ciblée du 15 octobre 2026.

[66] Projet LANCE. src/agent/evidence/ssh.py. Politique finie `ssh-crypto-policy-v1`, preuves d'algorithmes observés ; lecture du 15 octobre 2026.

[67] Projet LANCE. src/agent/evidence/mqtt.py. Attribution des messages au topic, format structuré et compatibilité historique ; lecture ciblée du 15 octobre 2026.

[68] Projet LANCE. docs/benchmark/v1/evaluation.md. Contrat courant de preuve, métriques, résultats indéterminés et absence de provenance de pivot ; consultation du 15 octobre 2026.

[69] Projet LANCE. src/agent/scanner.py. Extracteurs PKI et mises à jour signées, fonctions `_extract_pki_findings` et `_extract_ota_findings` ; lecture ciblée du 15 octobre 2026, sans sonde.

[70] Projet LANCE. src/agent/report_evidence.py. Admission au rapport et distinction entre échec d'exécution et preuve négative ; lecture du 15 octobre 2026.

[71] Projet LANCE. src/agent/evidence/capabilities.py. Règles d'accès observé, distinctes du contact réseau ; lecture du 15 octobre 2026.

[72] Projet LANCE. src/agent/phases/intrusion/observations.py. Projection des accès corroborés, contrôles de journal et transitions indisponibles ; lecture du 15 octobre 2026.

[73] Projet LANCE. docs/architecture/phase5-noop-gate.md. Gate de tentative observable du profil complet, conditions et limites ; consultation du 15 octobre 2026.

[74] Projet LANCE. src/agent/phases/intrusion/scope.py. Contrôle applicatif des destinations directes et commandes distantes ; lecture du 15 octobre 2026.

[75] Projet LANCE. src/api/auth.py. Authentification administrative, exceptions de lancement et sessions révocables ; lecture du 15 octobre 2026.

[76] Projet LANCE. docs/guides/admin-session.md. Session de huit heures, transport, réseau de confiance et limite à un processus ; consultation du 15 octobre 2026.

[77] Projet LANCE. tests/test_review_mqtt_ws_boundary.py. Tests des contrôles de périmètre, arrêt et budget de la sonde WebSocket. tests/pipeline/test_verification_evidence.py. Non-admission d'un upgrade WebSocket comme preuve d'accès MQTT. tests/test_intrusion_access_consistency.py. Cohérence des règles d'accès entre outil, livrable compact et projection ; lectures ciblées du 15 octobre 2026, sans réexécution.

[78] Projet LANCE. tests/pipeline/test_pki_identity_evidence.py. Contrôles positifs/négatifs et déduplication des équipements pour l'identité PKI ; lecture du 15 octobre 2026, sans réexécution.

[79] Projet LANCE. tests/test_review_mutation_auth.py. Tests du refus des mutations administratives et de l'exception des lancements publics ; lecture du 15 octobre 2026, sans réexécution.

[80] Projet LANCE. src/benchmark/metric_contract.py. Versions `strict-v3.16` et `evidence-v15`, empreintes des prompts et schémas d'outils ; lecture du 15 octobre 2026.

[81] Projet LANCE. src/api/main.py. Point d'entrée applicatif, routes et identité d'environnement ; état local consulté le 15 octobre 2026.

[82] Projet LANCE. src/api/routes/runs.py. Historique, statuts, artefacts et projection des diagnostics ; code consulté le 15 octobre 2026, sans vérification du service distant.

[83] Projet LANCE. src/static/app.js. Tableau de bord, graphe, synthèses et comparaison ; état local consulté le 15 octobre 2026.

[84] Projet LANCE. docs/architecture/run-modes.md. Modes d'exécution et frontière avec l'application ; contrat courant consulté le 15 octobre 2026.

[85] Projet LANCE. src/api/routes/pipeline.py. Validation, lancement, batch, arrêt et suivi ; code consulté le 15 octobre 2026.

[86] Projet LANCE. src/api/routes/providers.py. Contrat de configuration des fournisseurs ; code consulté le 15 octobre 2026.

[87] Projet LANCE. docs/guides/provider-auth.md. Authentification des mutations et fournisseurs exécutables ; guide courant consulté le 15 octobre 2026.

[88] Projet LANCE. docs/architecture/run-artifacts.md. Isolation des livrables et séquencement des appels au modèle ; guide consulté le 15 octobre 2026. Les validations historiques citées dans ce guide ne sont pas de nouveaux essais du présent bilan.

[89] Projet LANCE. src/api/run_coordinator.py. Coordination d'un run, état, arrêt et progression ; code consulté le 15 octobre 2026.

[90] Projet LANCE. src/api/events.py. Journal SSE borné et curseurs indépendants ; code consulté le 15 octobre 2026.

[91] Projet LANCE. src/api/routes/activity.py. Vue d'activité en lecture seule des instances connues ; code consulté le 15 octobre 2026, sans interrogation des instances.

[92] Projet LANCE. tests/test_review_batch_outcomes.py. Tests des statuts, décomptes et coûts de batch ; tests lus, non exécutés pour cette contribution.

[93] Projet LANCE. tests/test_api_run_modes.py. Tests des contrats de lancement et de coordination ; tests lus, non exécutés pour cette contribution.

[94] Projet LANCE. tests/test_rules_audit_ui.py. Tests de restitution des contrôles par règles ; tests lus, non exécutés pour cette contribution.

[95] Projet LANCE. tests/test_intrusion_observations_ui.py. Tests des projections d'accès et de l'absence de promotion des déclarations en preuves ; tests lus, non exécutés pour cette contribution.

[96] Projet LANCE. tests/test_pipeline_summary_ui.py. Tests des synthèses applicatives et des données indisponibles ; tests lus, non exécutés pour cette contribution.

[97] Projet LANCE. metric/squelette-latex/parts/partie3.tex. Mémoire de stage, chapitre QLoRA et HMoE ; dernière révision Git a98b480 du 28 août 2026. Provenances, volumes et pertes historiques rapportés ; artefacts GPU non réinspectés.

[98] Projet LANCE. model_training/configs/qlora_qwen2_5_3b.yaml. Profil 3B à quatre experts, hyperparamètres et feedback configurés ; copie locale lue le 15 octobre 2026, sans attestation d'exécution.

[99] Projet LANCE. model_training/train_qlora_3b.py. Chaîne SFT, séparation de validation, injection du feedback, chargement quantifié, sélection du checkpoint ; code lu le 15 octobre 2026, sans entraînement.

[100] Projet LANCE. model_training/templates/qwen2_5_tools_assistant_mask.jinja. Gabarit conversationnel Qwen et masques de génération des messages assistant ; lu le 15 octobre 2026.

[101] Projet LANCE. model_training/prepare_3b_datasets.py. Fenêtrage, catalogue d'outils, compactage et chaîne atomique de phase 5 ; code lu le 15 octobre 2026.

[102] Projet LANCE. model_training/preflight_3b.py. Contrôles de dépendances, données, CUDA et adaptateurs ; lecture du 15 octobre 2026, sans exécution GPU.

[103] Projet LANCE. src/agent/moe_server.py. Service compatible OpenAI, routage, verrou de génération et budgets de contexte ; code lu le 15 octobre 2026, sans constat d'état distant.

[104] Projet LANCE. docs/training/hmoe-openwebui.md. Procédure de service HMoE et d'accès privé ; consultée le 15 octobre 2026, sans connexion à une machine externe.

[105] Projet LANCE. docs/training/workspaces.md. Séparation du workspace canonique et du workspace d'entraînement, promotion et provenance ; consulté le 15 octobre 2026.

[106] Projet LANCE. model_training/workspace_sync.json. Liste blanche et limites de collecte des rapports et adaptateurs ; copie locale lue le 15 octobre 2026.

[107] Projet LANCE. docs/training/learning-loop.md. Minage des erreurs, revue humaine, attribution aux experts et exclusion des tests ; référence consultée le 15 octobre 2026.

[108] Projet LANCE. src/learning/sft_feedback.py. Conversion des corrections acceptées, contrôles de phase et d'origine ; code lu le 15 octobre 2026.

[109] Projet LANCE. src/learning/remote_runs.py. Import contrôlé d'artefacts et manifeste de provenance ; code consulté le 15 octobre 2026, aucun import exécuté.

[110] Projet LANCE. model_training/README.md. Périmètre des outils et limites des tests sans entraînement GPU ; consulté le 15 octobre 2026.

[111] Projet LANCE. benchmarks/scenarios/dev/S15.yaml. Composition S15, lue avec la topologie, la vérité terrain scenario_15.yaml et le pack f11_api_authorization.yaml ; descriptions statiques, sans nouvelle vérification du laboratoire.

[112] Projet LANCE. docs/benchmark/derived-index.md. Origine et limites des axes calculés depuis les vérités terrain ; consultation le 15 octobre 2026.

[113] Projet LANCE. objectifs/asiaccs-2027/audit-couverture.md. Audit statique daté du 12 octobre 2026 ; 288 vulnérabilités attendues et proposition de pilote, sans run.

[114] Projet LANCE. src/benchmark/funnel.py. Entonnoir candidats/retenus/confirmés et disponibilité des populations ; lecture statique du code local.

[115] Projet LANCE. docs/benchmark/agent-vs-automation-cli.md. Périmètre D1/A1 des phases 3–4, outils et calibration SSH ; la mention ancienne d'une inférence absente est confrontée au code courant.

[116] Projet LANCE. src/agent/audit_experiment.py. Périmètre des expériences sur inventaire et version shared-scanner-bounded-verification-v5 ; lecture statique du code local.

[117] Projet LANCE. src/agent/phases/verification/rules.py. Règles de reprise bornée, observations et validation de preuves communes ; lecture statique au 15 octobre 2026.

[118] Projet LANCE. docs/benchmark/agent-vs-automation-campaign.md. Manifestes, issues opérationnelles, ressources, archivage et revue indépendante ; consultation locale au 15 octobre 2026.

[119] Projet LANCE. src/benchmark/compare_policies.py. Critères d'admissibilité d'une paire et export des motifs ; aucun comparateur exécuté pour ce rapport.

[120] Projet LANCE. src/benchmark/policy_campaign.py. Bilan opérationnel, regroupement des configurations et appel de l'inférence multi-familles ; lecture statique du 15 octobre 2026.

[121] Projet LANCE. src/benchmark/inference.py. Bootstrap par famille, intervalle à 95 % et Holm entre métriques ; implémentation présente, sans application à une campagne admissible dans ce bilan.

[122] Projet LANCE. objectifs/asiaccs-2027/plan-soumission.md. Plan de soumission retenu le 12 octobre 2026 et critères en vrais positifs confirmés ; objectif à aligner aux tâches V2, sans résultat revendiqué.

[123] Projet LANCE. research/2026-10-05-benchmark-coverage/README.md. Premier lot Qwen UMONS commencé le 4 octobre ; données gelées le 5 octobre 2026 à 11 :32, rapport révisé à 11 :48 ; valeurs historiques transcrites sans recalcul.

[124] Projet LANCE. research/2026-10-05-corrections-lance/run-01.tex. Rapport Run1 révisé le 5 octobre 2026 à 23 :38 ; scores et statuts historiques, réserves sémantiques préservées.

[125] Projet LANCE. research/2026-10-05-corrections-lance/run-02.tex. Campagne corrective distincte, révision du 6 octobre 2026 à 00 :49 ; lecture de métadonnées sélectionnées.

[126] Projet LANCE. research/2026-10-05-corrections-lance/diagnostic-pipeline.tex. Diagnostic hors ligne révisé le 6 octobre 2026 à 10 :34, inventaire de 09 :29 ; 2 done, 8 partial, 2 failed ; sans certification exhaustive des preuves.

[127] Projet LANCE. benchmarks/experiments/policy-comparison/pilot-2026-10.json. Manifeste réel lu directement le 15 octobre 2026 : 12 paires prévues, 24 run_dir nuls et précontrôles pending ; configuration et budgets consignés.

[128] Projet LANCE. benchmarks/experiments/policy-comparison/pilot-2026-10-preflight.md. Journal des préparatifs et fumées des 13–14 octobre 2026, calibration et changement de gel ; nombres attribués au journal, sans nouveau calcul des traces.

[129] Projet LANCE. research/2026-10-12-recentrage-corpus/README.md. Plan V2 révisé le 15 octobre 2026 à 14 :49, proposition sans résultat expérimental nouveau : 44 fiches, 25 ensembles, 216 configurations et 456 bras conditionnels.

[130] Projet LANCE. research/2026-10-12-recentrage-corpus/coverage-plan.json. Plan structuré proposé, contrats, tâches et comparateurs ; aucune exécution déduite des nombres de configurations.

[131] Projet LANCE. benchmarks/experiments/ssh-reference/README.md. Premier incrément SSH et qualification logicielle consignée le 15 octobre 2026 ; SSH simulé, fixture réelle et A1 restant à réaliser.

[132] Projet LANCE. benchmarks/experiments/ssh_reference.py. Contrat public, adaptateur D1 et évaluation probatoire SSH ; lecture du code courant, sortie non admise comme mesure M-T.

[133] Projet LANCE. research/2026-09-29-context-scalability/README.md. Étude active, diagnostic et reproductions du 29 septembre 2026 ; impact réel non mesuré.

[134] Projet LANCE. research/2026-10-07-methodes-audit/README.md. Exploration des méthodes et protocole proposé, révision du 7 octobre 2026.

[135] Projet LANCE. research/2026-10-03-organisation-code/README.md. Diagnostic historique et index mentionnant une mise en œuvre partielle ultérieure.

[136] Projet LANCE. research/2026-09-29-iomt/README.md. Proposition IoMT non démarrée ; date de classement distincte d'une date de début connue.

A Repères de vocabulaire

Ce tableau rassemble les principaux termes employés dans les parties spécialisées. Une confirmation désigne une déclaration soutenue par les observations admises au contrat applicable. Elle reste limitée à la propriété, à la cible et aux conditions effectivement contrôlées.

Terme	Sens dans le rapport
API	Interface de programmation reliant le tableau de bord, le contrôleur et les services.
Benchmark	Banc d'essai avec configurations et critères de mesure identifiés.
Bras	Exécution d'une politique ou d'un traitement dans une comparaison.
CIDR	Notation d'une plage d'adresses réseau utilisée pour déclarer le périmètre.
CPE / CVE	Identifiant de produit ou de plateforme / identifiant d'une vulnérabilité publiée.
CVSS	Cotation de sévérité ; ne mesure pas la probabilité d'une compromission.
D1 / A1 / A0	Automatisation conditionnelle sans LLM / politique LLM révisable / plan initial LLM figé. A0 est proposé ; D1/A1 dispose d'un incrément logiciel.
Ground truth	Vérité terrain : propriétés attendues réservées à l'évaluation.
HMoE	Combinaison de rôles spécialisés via adaptateurs, routés par phase au niveau de la requête.
Instance	Selon le contexte, processus applicatif ou monde expérimental paramétré. Dans une paire de benchmark, les politiques reçoivent les mêmes paramètres de monde.
IoT / OT / IoMT	Objets connectés / systèmes opérationnels industriels / objets connectés médicaux.
LLM	Modèle de langage de grande taille, employé pour certaines décisions et explications.
LoRA / QLoRA	Adaptation par paramètres supplémentaires de faible rang / entraînement de ces adaptateurs avec base quantifiée.
MQTT / topic	Protocole de messagerie publish/subscribe / canal de messages.
NVD	Base de vulnérabilités utilisée pour l'enrichissement, ou via un instantané en benchmark.
OTA	Mise à jour à distance ; le verdict reste limité aux propriétés contrôlées.
Pack / posture	Ensemble de propriétés composables / profil de configuration d'une instance.
Pivot	Action depuis un accès acquis vers une autre cible, avec causalité de la transition réseau à démontrer.
Preflight / reset	Vérifications avant audit / procédure de remise en état entre essais.
Run	Exécution identifiée, avec son dossier, ses contextes, ses observations et son état terminal.
SSE	Flux d'événements du serveur vers les lecteurs pour le suivi d'exécution.
SSH / TLS	Connexion distante chiffrée / protection du transport et authentification selon configuration.
VLAN	Séparation de domaines réseau ; routage et filtrage demandent des règles distinctes.
VP / FP / FN	Vrai positif / faux positif / faux négatif, définis selon la population et la vérité terrain.